Latches

Luis Oliveira

Original slides - wilkie
(with content borrowed from: Jarrett Billingsley and Bruce Childers)

CS 0447
Introduction to
Computer Programming

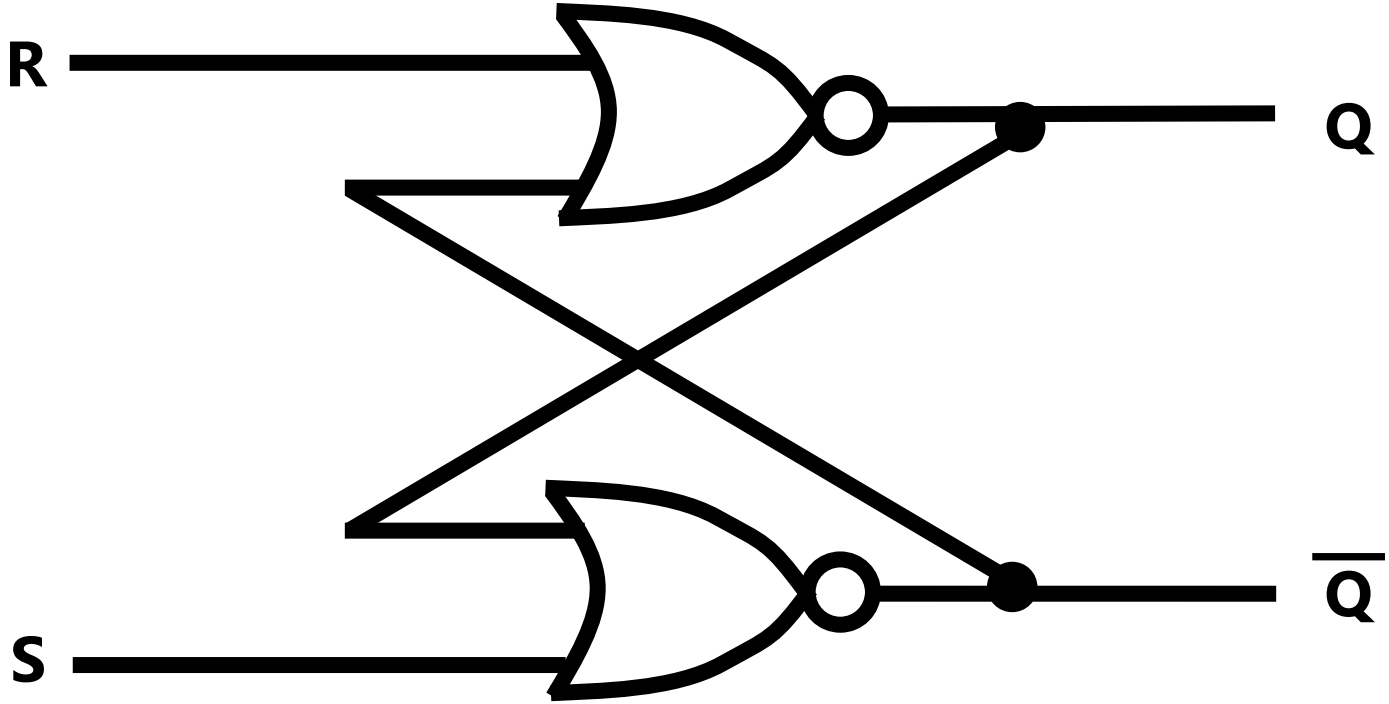
Luís Oliveira

Announcements

The Latch

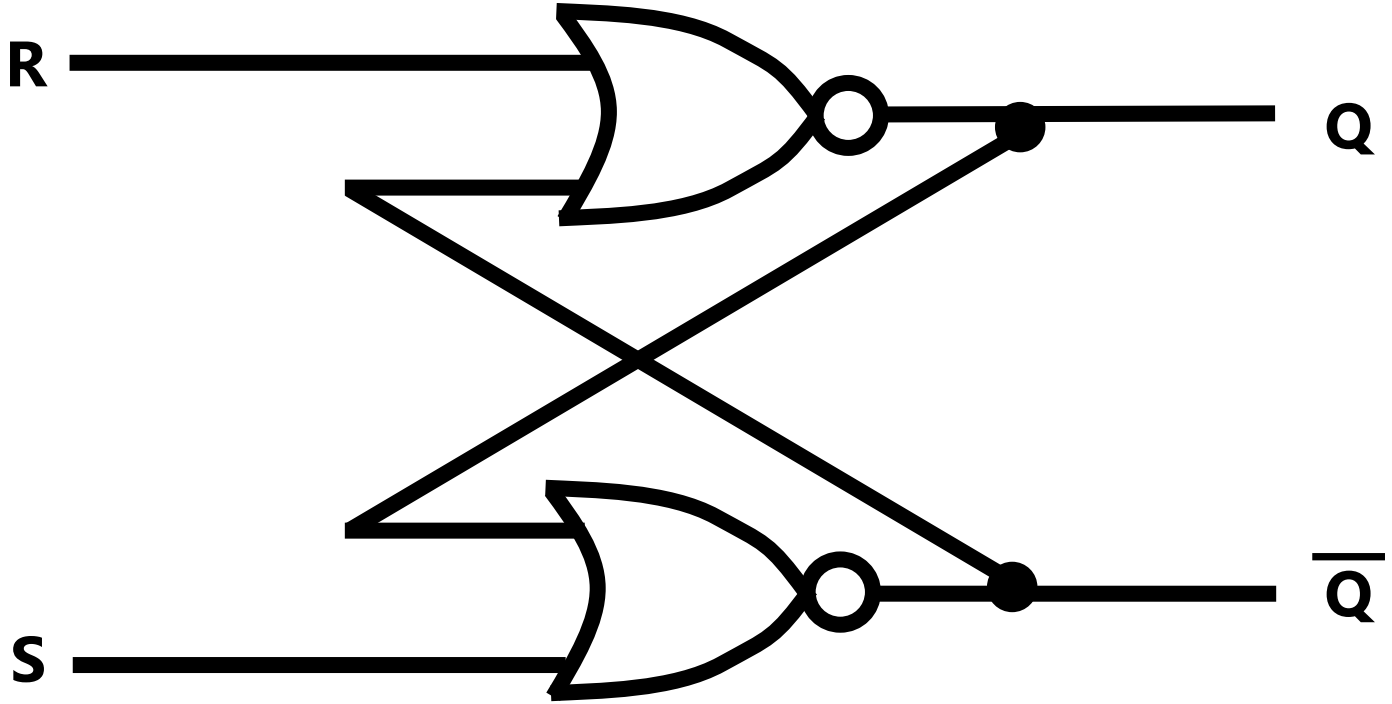
What the heck

- Time to blow your mind.
- Let's look at this circuit.
  - Yes, you can do this.



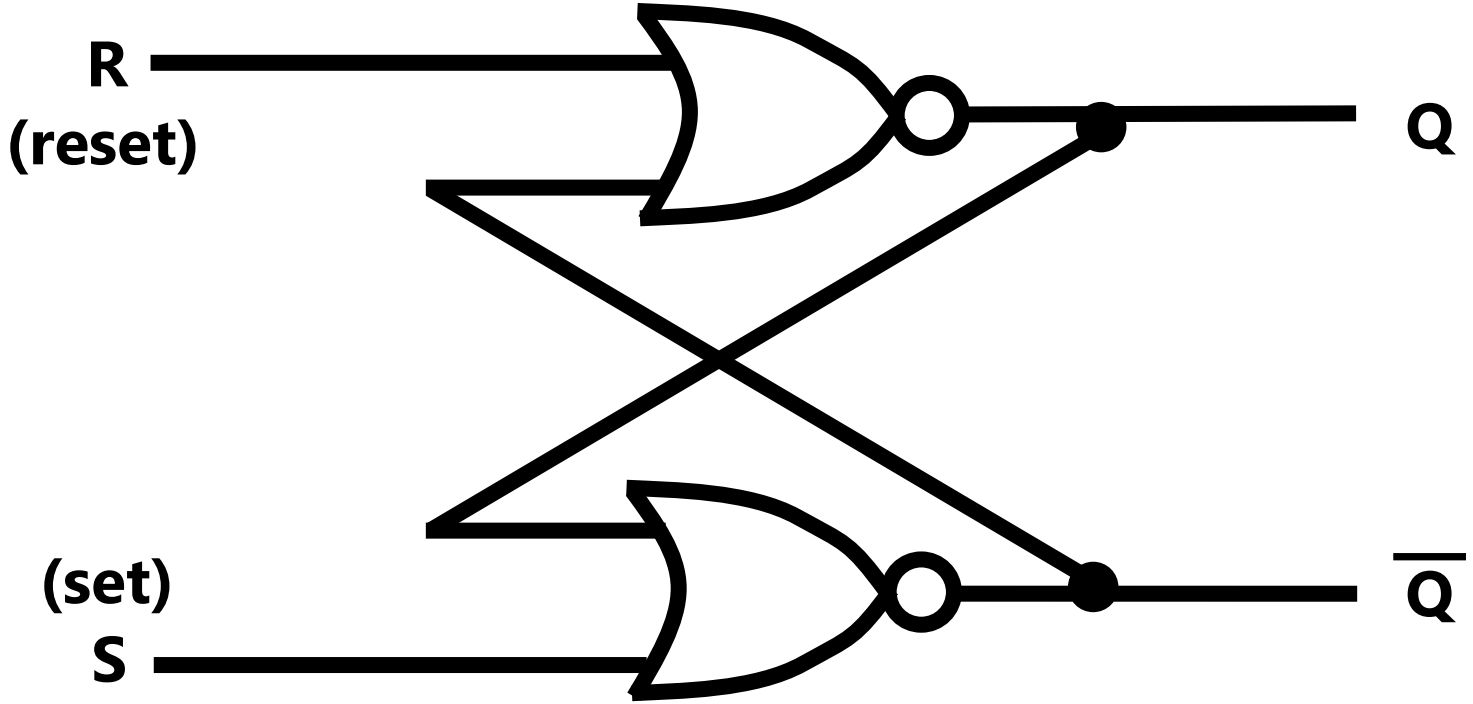
What the actual heck

- What is this in combinational logic??
  - Q = R + (S + (R + (S + (R + (… oh gosh. Hmm
- This makes no sense (from a combinational point of view)



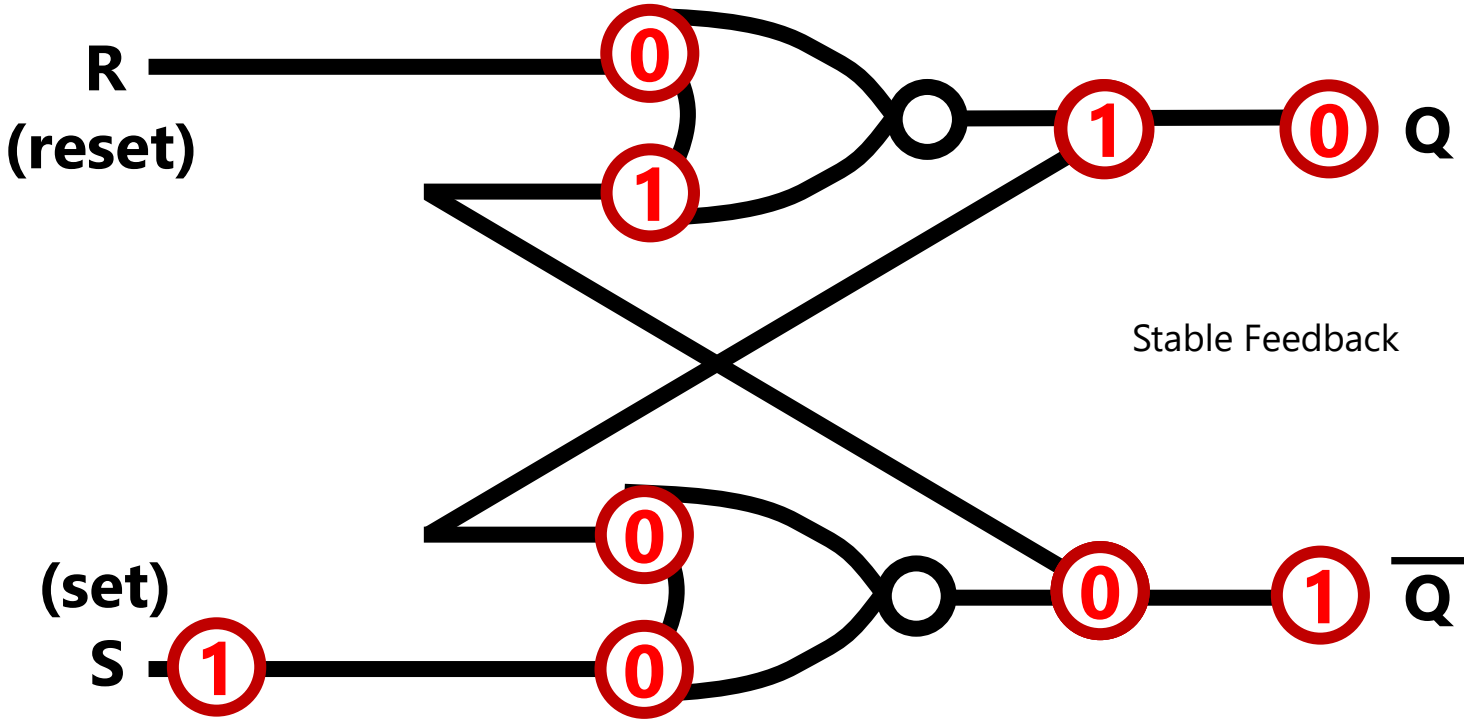
S/R Latch

- The feedback behavior allows it to “store” a value.
  - “1” on S will set Q to “1”, and “1” or R will reset Q to “0”
  - It becomes stable at that value, hence, it is an S/R Latch.
    - It’s really heckin’ neat.



S/R Latch (animated)

- Currently, the value is “0”, and we can change it to “1” by sending a “1” on S

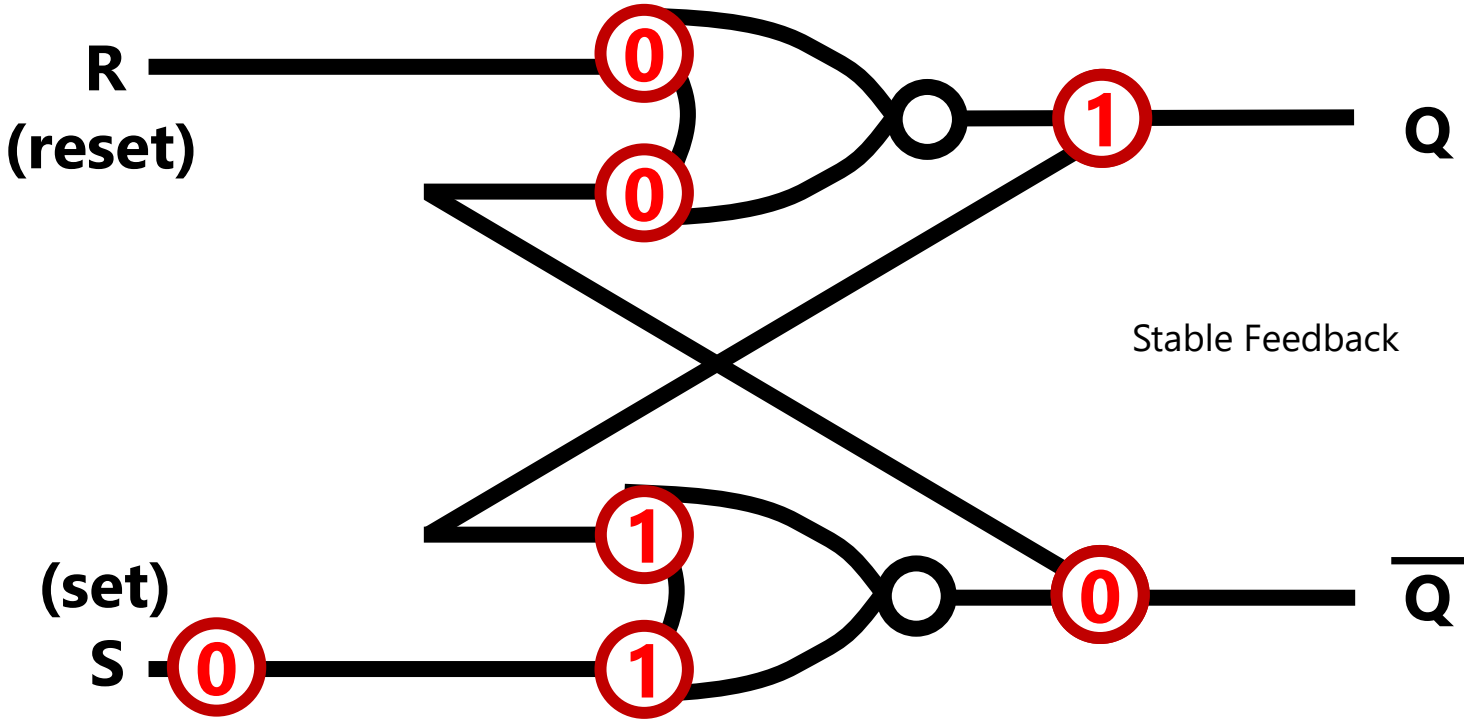


NOR

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

S/R Latch (animated)

- Currently, the value is “1”, and we can maintain it by having R and S be “0”
- If the value was “0”, this would also maintain that value.
- This is the “latch” operation. This is how an S/R Latch can “store” a value.

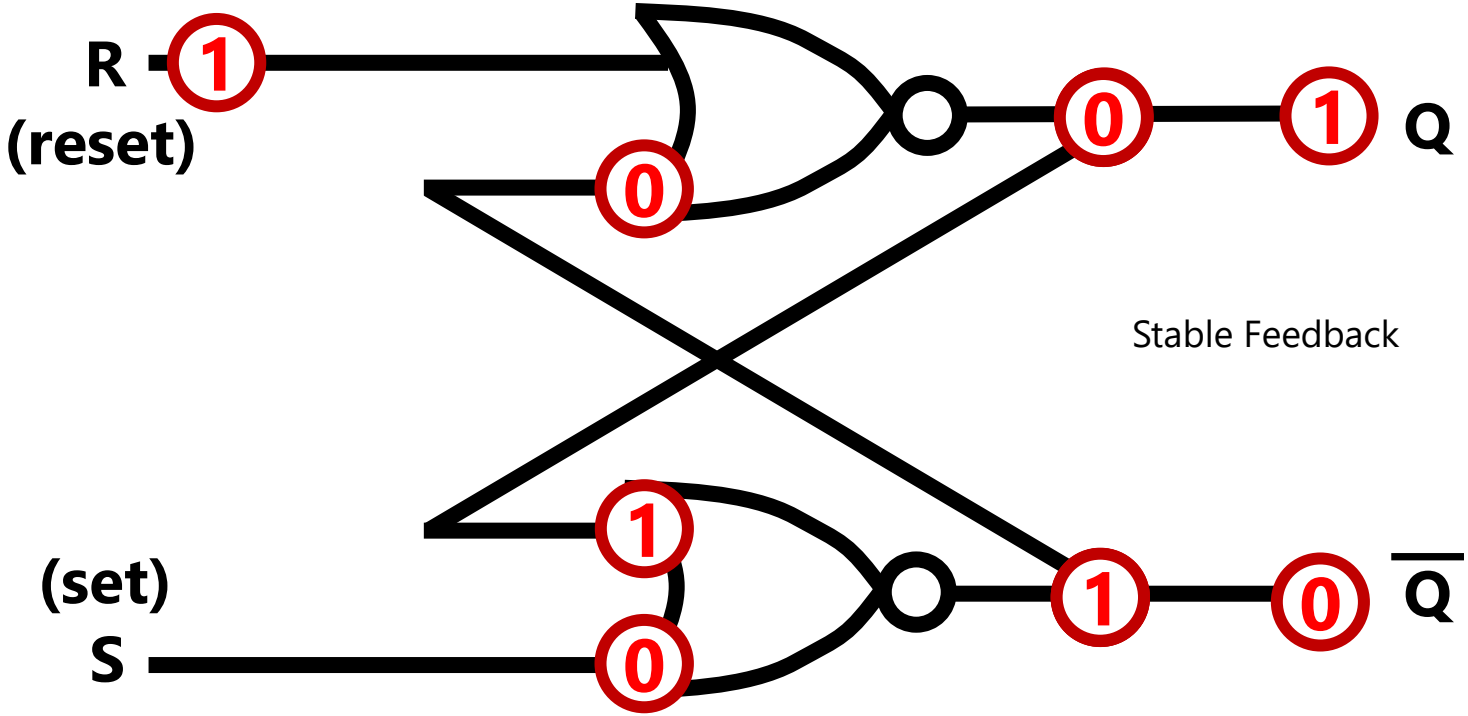


NOR

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

S/R Latch (animated)

- Currently, the value is “1”, and we can change it to “0” by sending a “1” on R
  - And having S be “0”

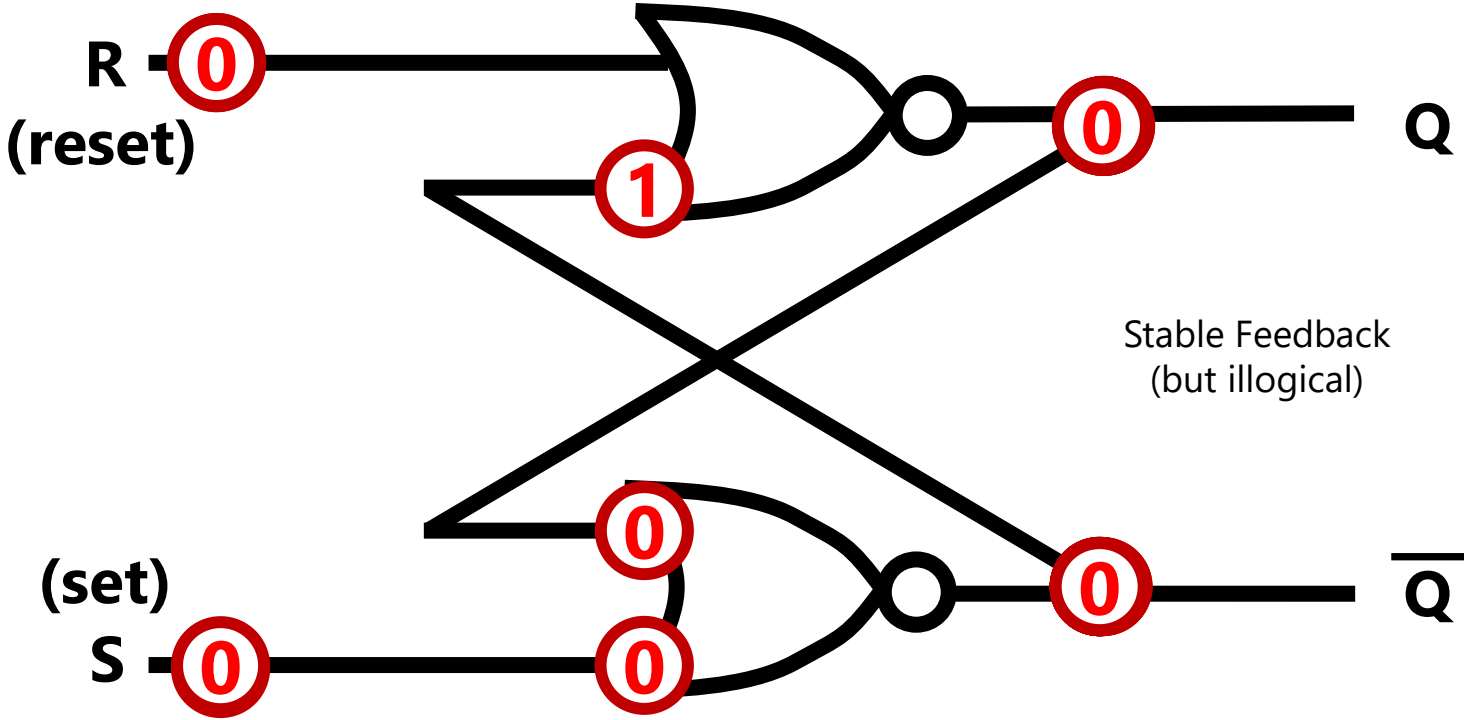


NOR

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

S/R Latch (animated)

- Hmm, let's set R and S to "1" at the same time.
- And then let's set them both to "0" afterward. (That should be stable… right?)
- Oh no. *Oscillation*. Q is… both… 0 and 1… ??? Kinda???

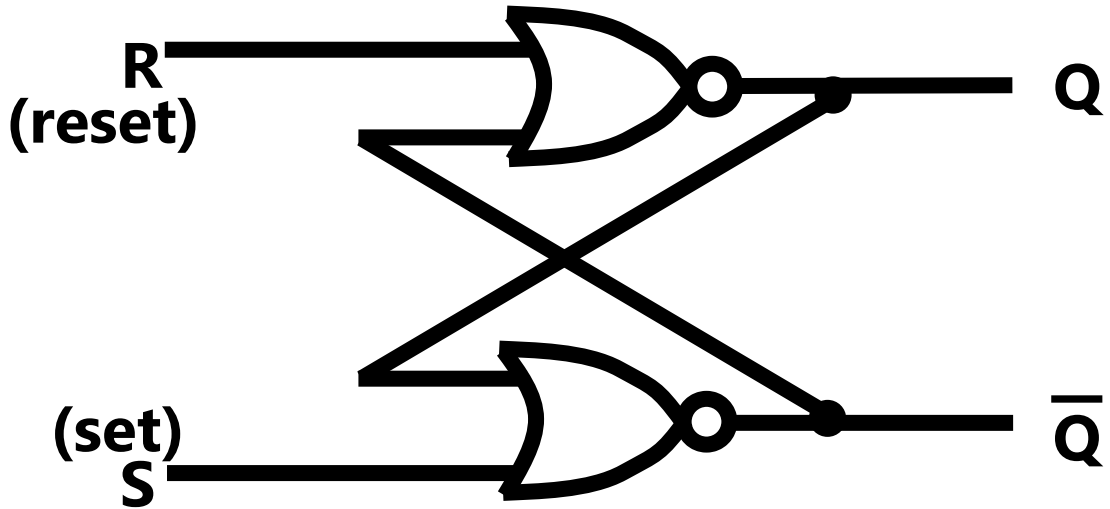


NOR

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

S/R Latch: The Whole Truth table

- The state of this logic depends on the prior state.
  - Q here is the current value of Q
  - Q_{next} will be the new value.
- This is an example of **sequential logic**.
  - **On your own**: You can build it out of NANDs as well. Try to come up with that.



Q	S	R	Q_{next}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	oh no
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	aaaa

Let's look at some circuits

- sr_latch.circ

- Factorio latch

0eNrtV8GOmzAQ/ZVqzlCB2ZANUvsFPbXHqkIOTJKRjEHGREUR/14bWjYb1iyhP
WylPYTI2J558948AxfYiwYrRVJDcgHKSllD8v0CNR0lF/aebiuEBEhjAR5IXthRjhnlqP
ysLPYkuS4VdB6QzPEnJGH3wwOUmjThEK0ftKlsij0qs2CMY/NpLvV1lA+qsjZ7S2n
Tm3h+5EFr/kKTwmzQqhTpHk/8TGa1WXIgoVE5cJ9J6cbcGVMOK3yFOfTxGlt6eA
1+SCMxsxhqGyu0l6NClNflUA4JM2tJZQ3pfmh3d503qZjNMTctOPi4eSo5JzVAGb
K9QMDvmKmZy2kEfSBV63QxIUN5Q+29JpAEdlBUXPUwE/hkNpWNrpp1Yas27el
OD6osUpImDiQHLmrsHJRbkW4Ij24J955Nx8+nmW3FF4W73RhNhfSALQPB5kG
4miJa2RTsTTXF5xVNMZrvn7TE1IPeYrO6NWbzGm8coj481VpwlXzBi2oqZjh/ov1J
PGq3QjouW30ieXxNPTPd1GhSidKeoVo1uJx6FwubEU2BOTWFj8JEU5T5VSnQyQ
dbfQ7cyvPgABav81z4brl7zLFb6sjdXzjy9gTfzoNgjo7Y3tmqbGylhVzF81yFgQPY4
6q3JPYGX5J2y56Hu3XeZO/vSPMd57ZfPO+ox1k7xsv8FQZ3PBHZ//5A3PYsGL777
6Xk6vPKA8H3aIDCV//bhy9cZydz72w82ZezjYJwE8X213W/ABNorVw=

Let's look at some circuits

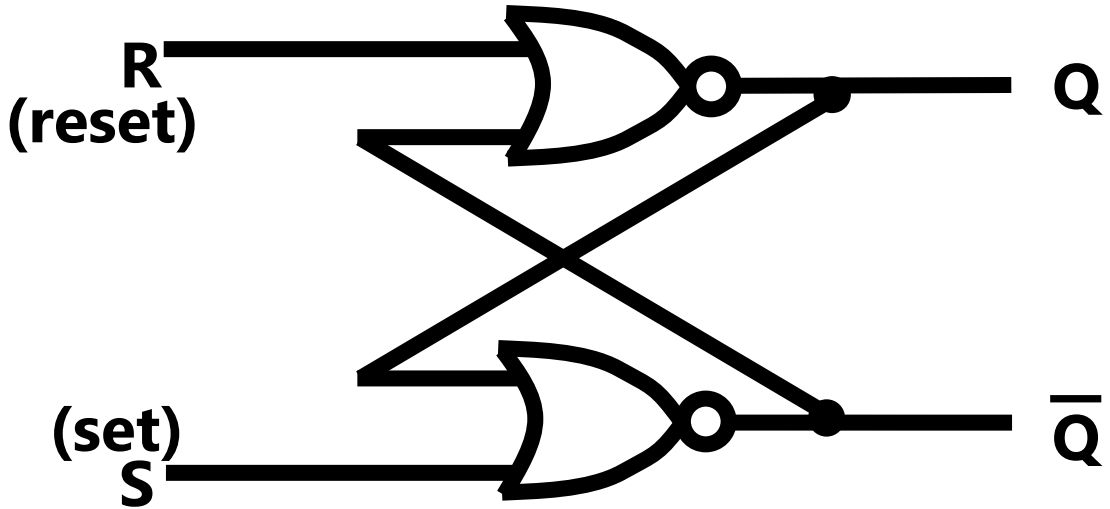
- Factorio power latch

0eNrtWM2OmzAQfpVqzlBh8o+0K/XeQ+9VhRyYbCwZGxmTNop4gL5Hn6xPUh
u6bBrCbyMlhz2EyMb+5ht/M2ObE2x5jqliQkNwAhZJkUHw9QQZexGU2z59TBEC
YBoTcEDQxLZijFiMyo1ksmWCaqmgcICJGH9AQIpvDqDQTDOs0MrGMRR5skVlBt
Q41p6mQp8DOZDKzMyVwpo3eO7MgaP9MybMBK0kD7e4pwdmRpshO8Y1qh
beB6Z0bnpqk9UI9xOUaLl1nHjeOfnKjMDIcsgsFrEPhfG5M8y0DLGIqShnumzauU
XhNPz1u9at6S75uKgcJoZJzFRFpIS55v5fzNC8i1lNecdUpsOxy1GpAcFmY1tJSlXJ
MYBnM0PmOs1HYL4oRFHhpsewXOtwp2QSMmFwINhRnmHxH6vt/PN61RTDAd
9iVUS60fwLtPkwaWcTpfUfR9rfP39NENfqcztp/W5p+7T0L7W8LtZ8lFjeHdLwLGVe
9flu5Hp6gExcdMu1bsjhDNTN6TZTR0E/p0usZTfHtvxeTAsZ/6FC5vnu+d2spi0RM
R8j+XxY2i9r3xKMWZ64yA1lxSI3lRybKvpvaT/NvcvY27QQW03bPO53LCDeTfaOW
5ejvt3DG3cyWI+Dawu79aTSQd4rx5jUImRoZVmNBhpeeUgPWFuIbN7WLqGcu5
wmaVs98lpuP6926lCYEAlUHPWeiZe+YDCv8wyNKS7tfUurHlcruWxZBJtPU8qg/z
hl8P5pMvLysxoWn4RMqmH++4G5b0tpq1LrMVVofU1Fg1x+qwnOPu04wOkWje
PwRX5H9eEz1dHe9B5QZaVEq5lHFrOl/RXFHz5GPCg=

The Clock

Propagation Delay (Basics)

- Ok. Q at t=0 is different than at t=1.
- How long does it take for a change to occur?
  - (How much time is really between t=0 and t=1)
- This is bounded by **propagation delay**.

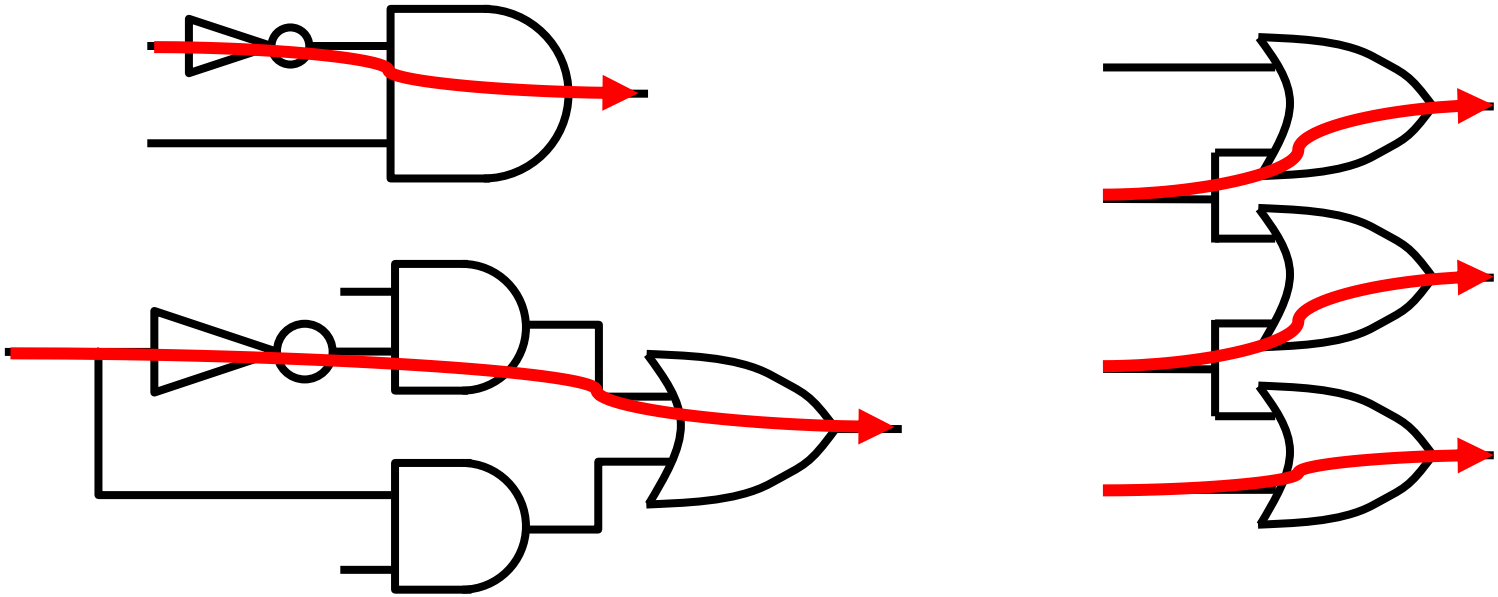


Propagation Delay

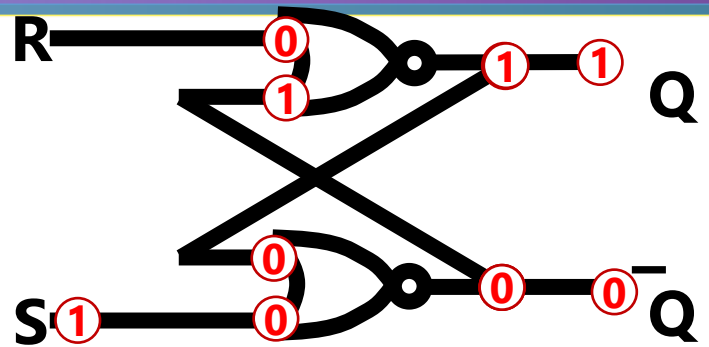
- **Propagation delay** is the time it takes for a signal to pass **from the inputs to the outputs**
- During that delay, the outputs are **invalid** (they can fluctuate)
- After that delay, the outputs are **valid**
- If you try to use the output *while it's invalid,* **things break**
  - stuff like 2 + 2 = 17??

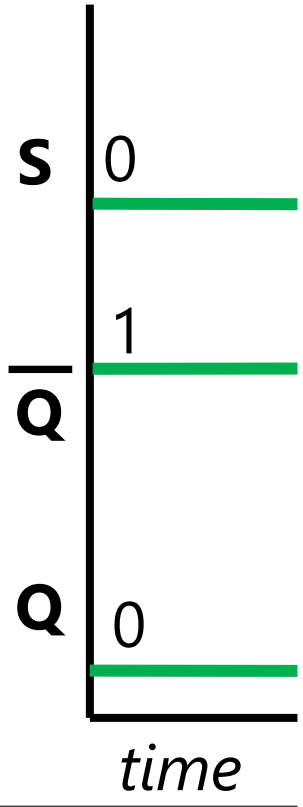
The Critical Path

- The **critical path** is the path through a circuit that has the longest series of **sequential** operations
- The longest **propagation delay**
  - they depend on each other and **can't be done in parallel!**



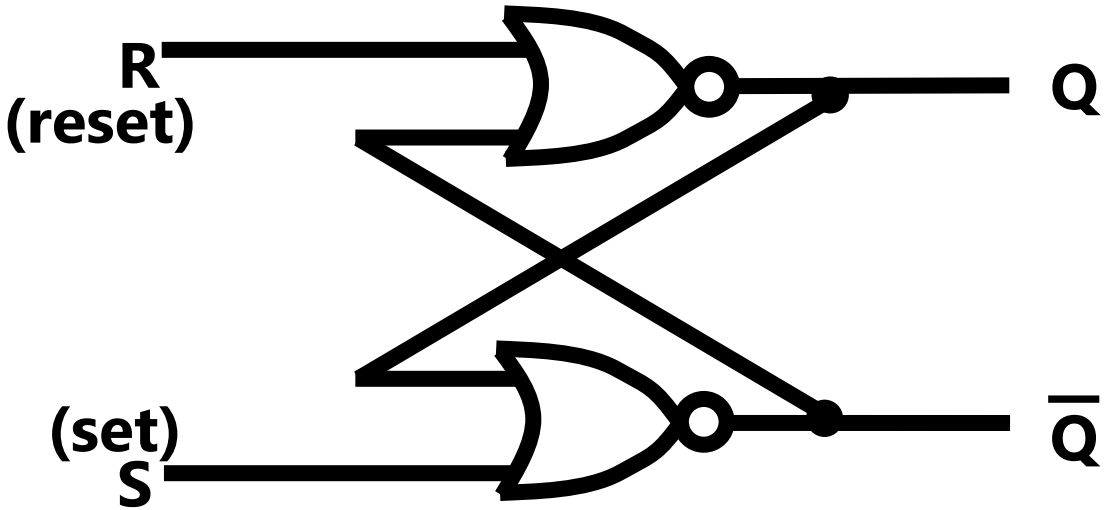
S/R Latch (animated)





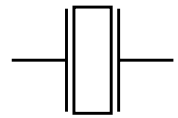
Propagation Delay (Basics)

- If we have a component after this S/R latch that reacts to the data on Q… we need it to **synchronize**.
  - We need it to *wait* until the Q value is updated.
- One method: something that periodically and predictably updates in an interval that's a little longer than the propagation delay.

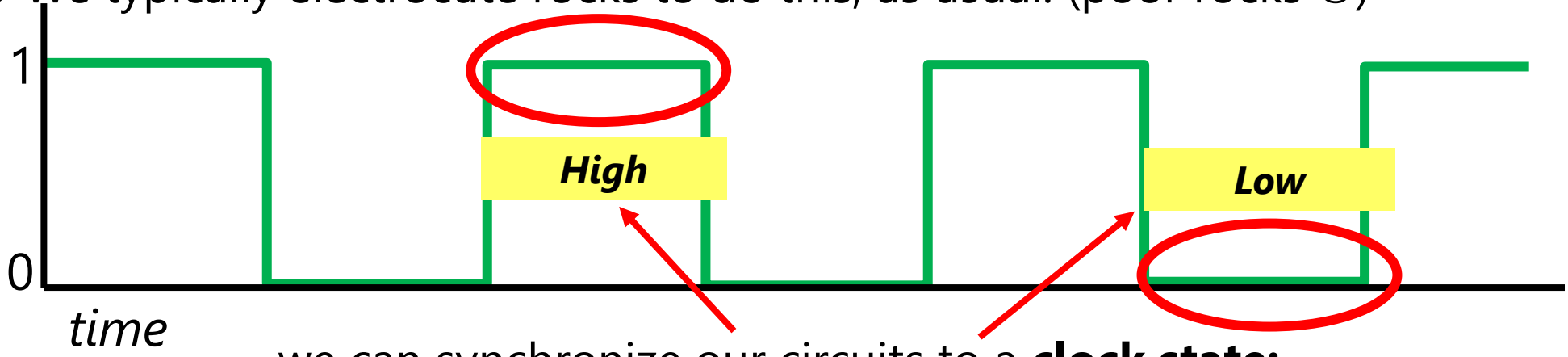


Tick Tock

- Sequential logic is based on time, and time is *continuous*
- Trying to build sequential circuits without anything to keep track of time is… possible, but very very difficult
- This is why we use a **clock signal:** it goes 0, 1, 0, 1, 0, 1…
  - Oscillation… *on purpose this time.*
    Each period is called a **clock cycle**.
  - We typically electrocute rocks to do this, as usual. (poor rocks ☹)



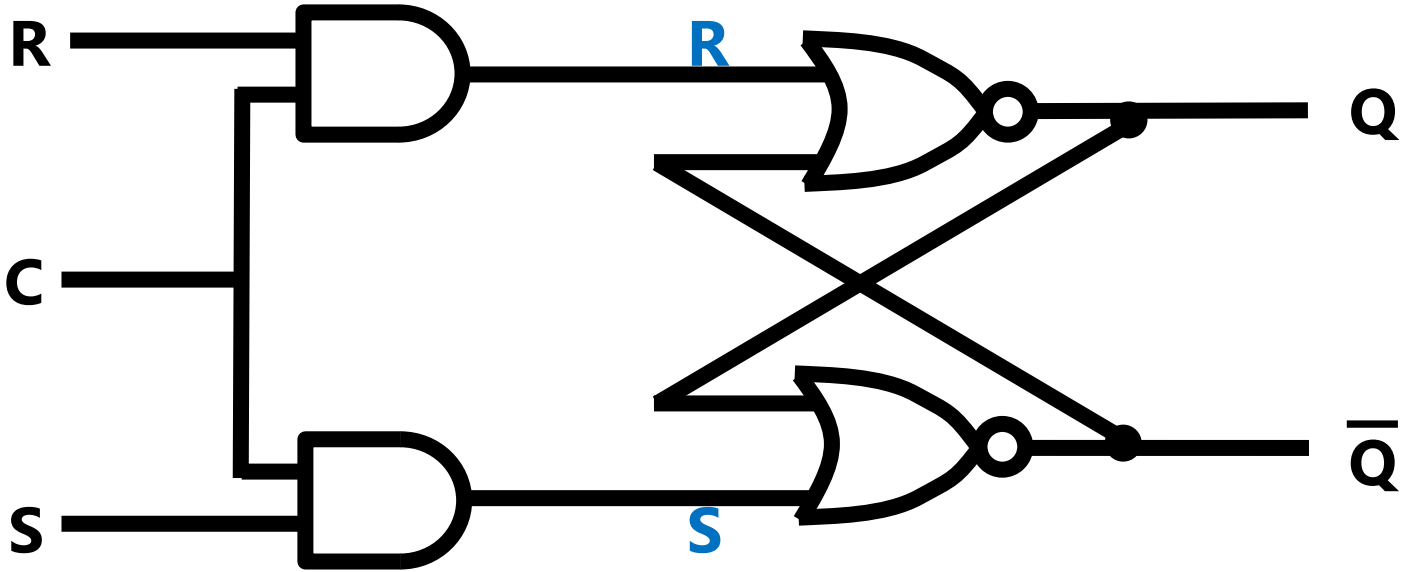
circuit symbol



we can synchronize our circuits to a **clock state:**
when it is high (1) or low (0)

Adding a Clock

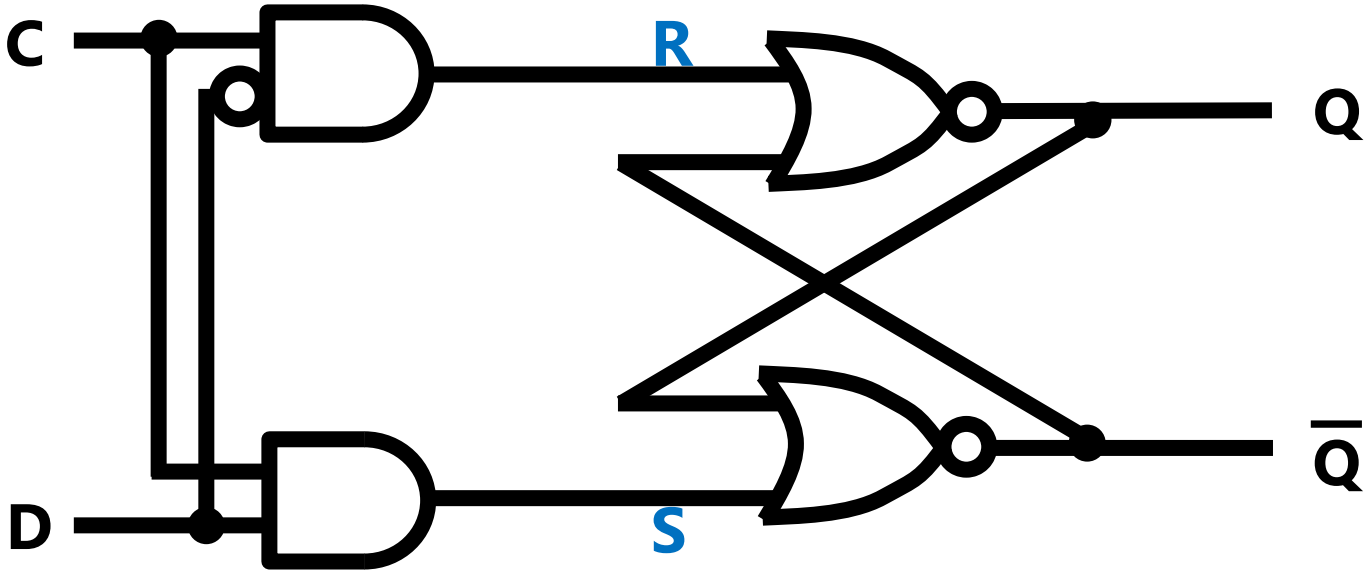
- We need to augment our latch to wait for the clock before updating the value.
- We need to account for the clock signal and only transmit a “1” on R or S if and only if the clock is high (or low).
- Which leads us to something like this maybe…
  - But let’s refine it a bit… (btw, how complex is this? How many transistors?)



D(ata) Latch

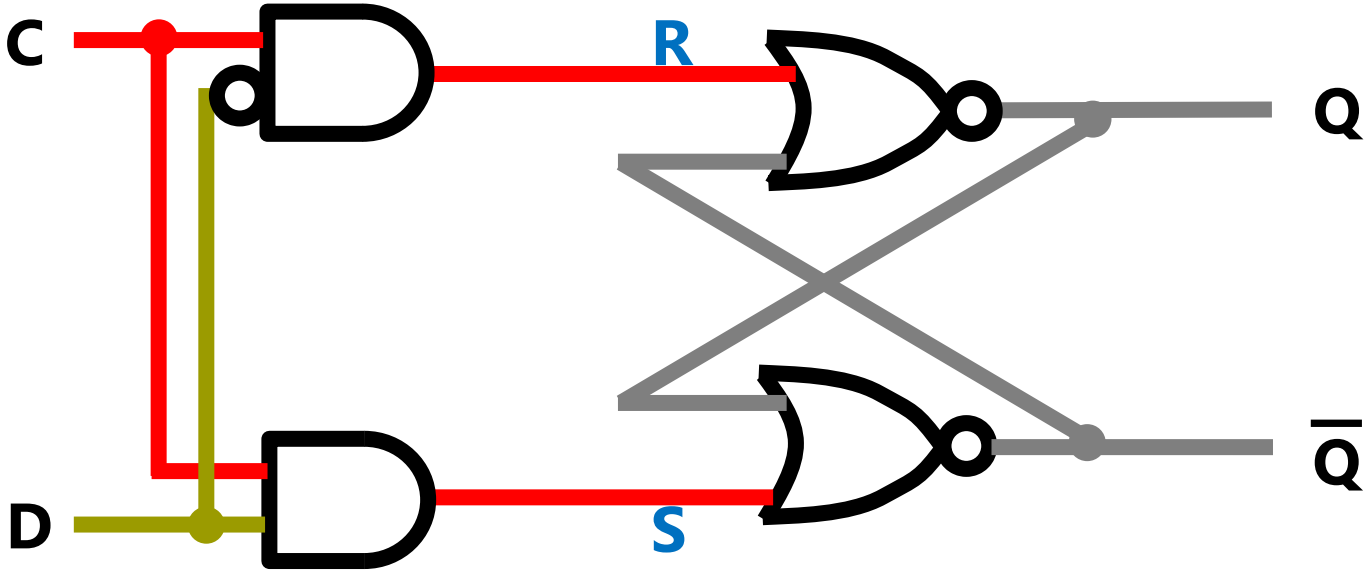
- If we do something like this, we simplify our S/R Latch into a nicer synchronized latch called a D-Latch.
- "C" is the clock. "D" is the data to latch when the clock is high.

- Circuit only changes the value when the clock signal is 1…
  - Latches when clock is 0!



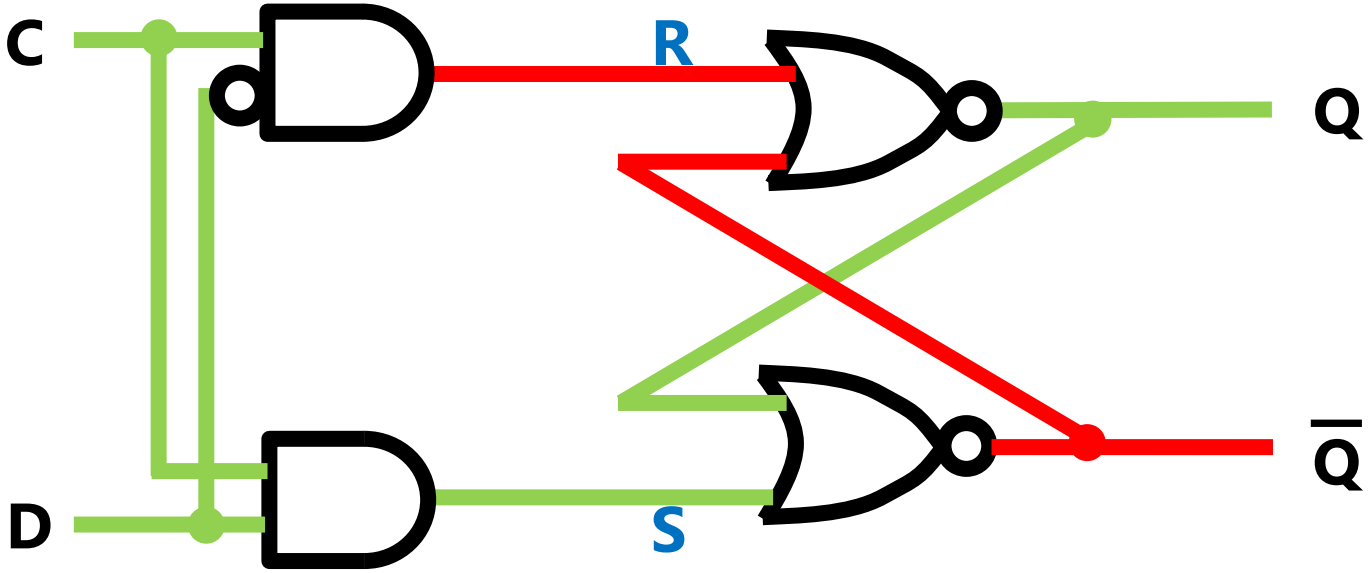
D Latch

- When clock is low and D is … don't care!
  - Nothing changes



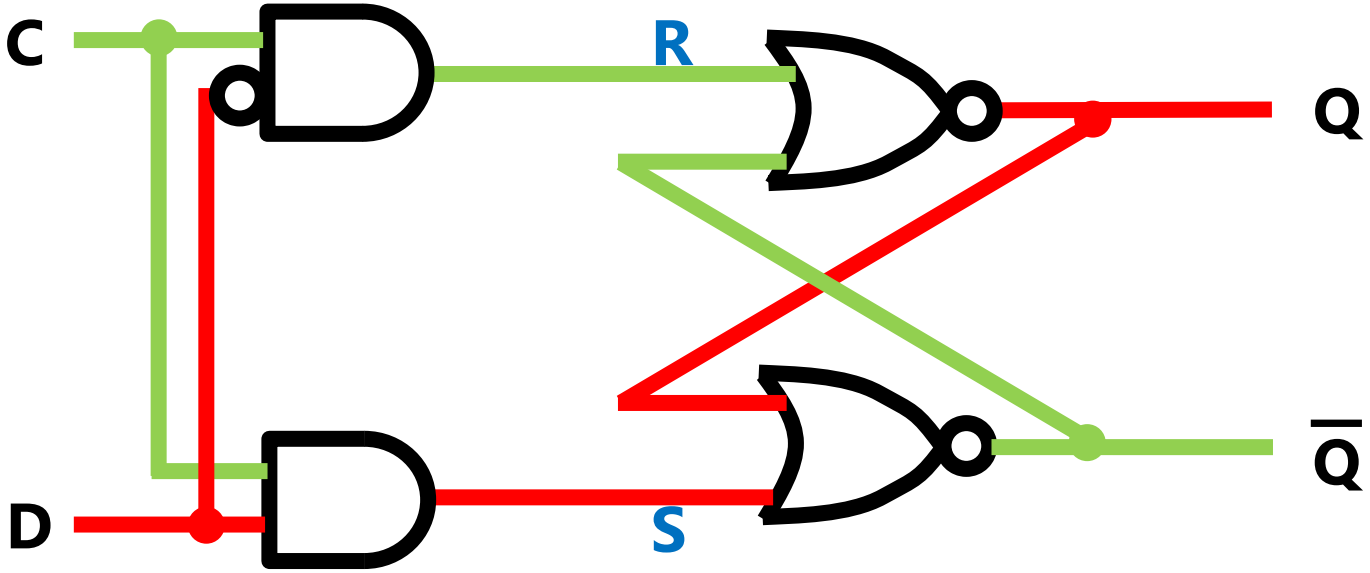
D Latch

- When clock is high and D is high
  - It's a set operation



D Latch

- When clock is high and D is low
  - It's a reset operation



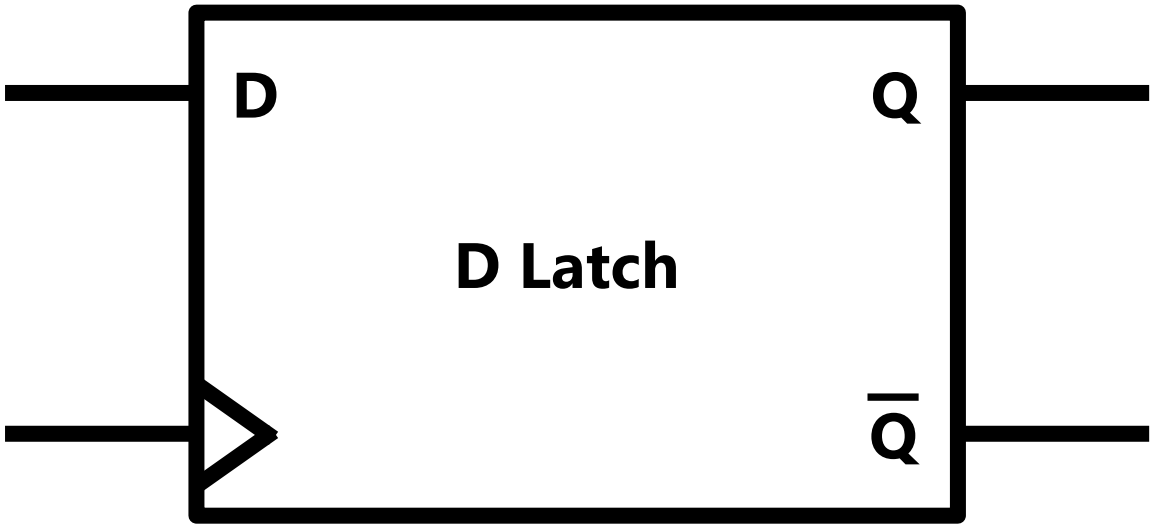
D Latch

- We can abstract this away and start using this symbol:



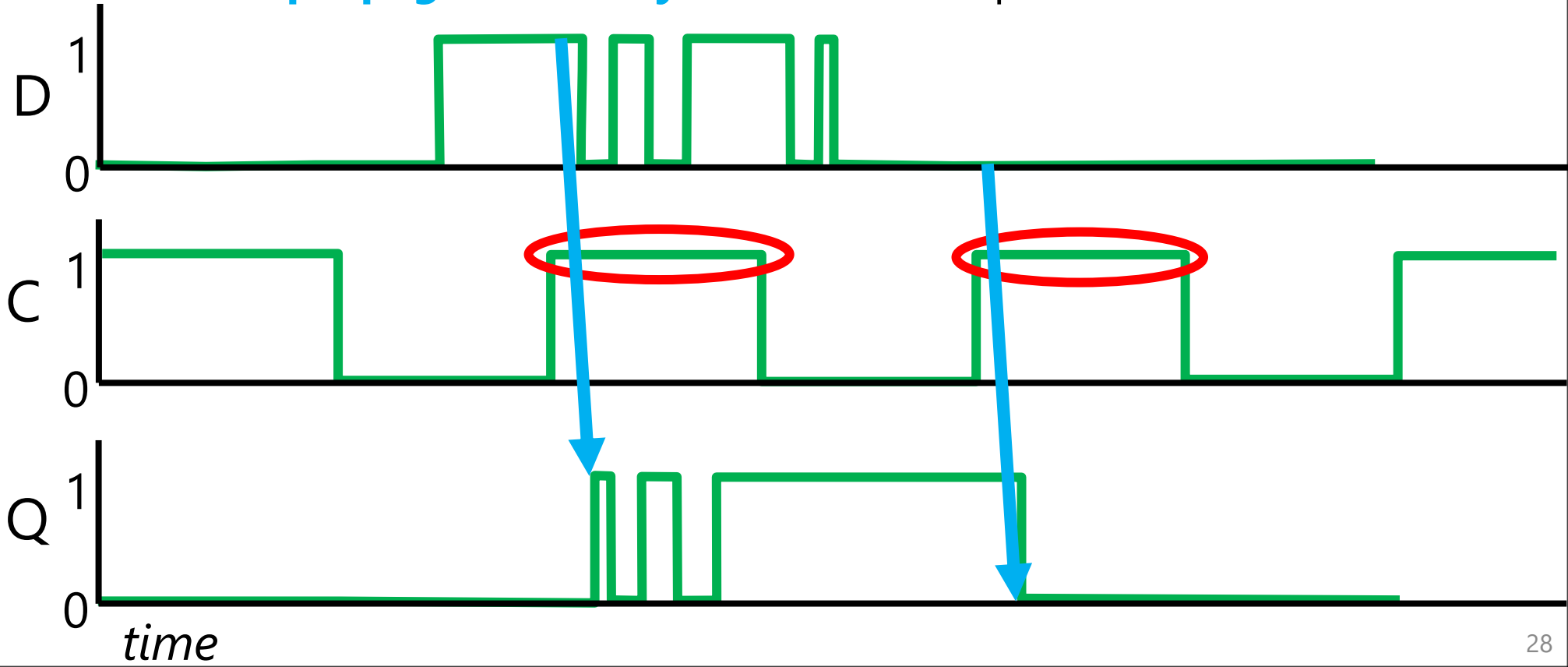
D Latch

- We often omit the "C" for the clock and use a triangle instead:
  - Sometimes you'll see a square instead. Logisim uses triangles.



Tick Tock: D Latch

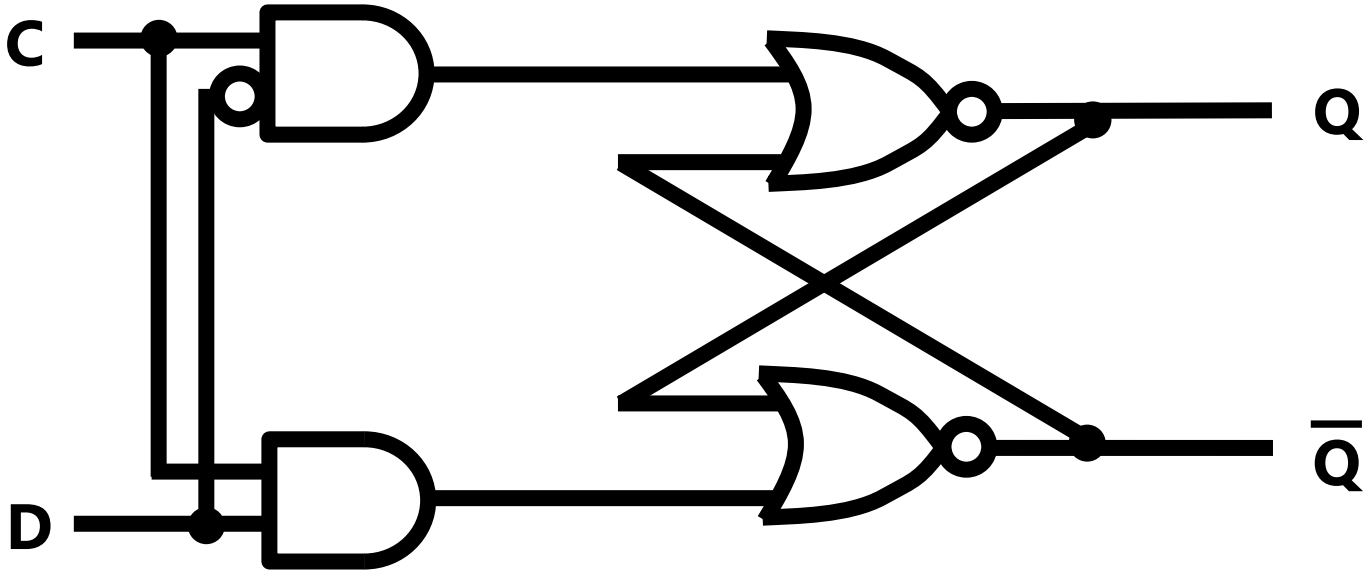
- This diagram shows the behavior of the system over time.
  - This is "high" triggered.
  - Note the **propagation delay**. And how Q depends on D AND clock.



Problems: Owner of a lonely (D) Latch

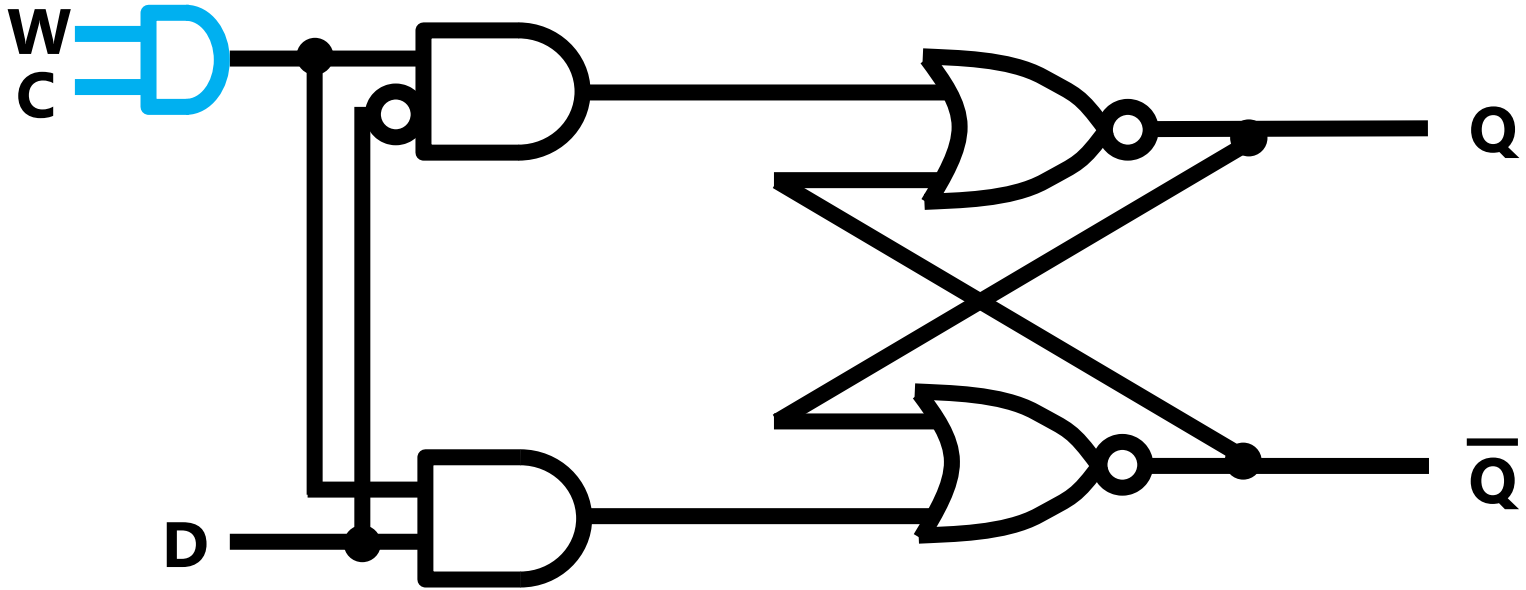
- What if we don't want to change the value of Q.
  - This means we have to constantly recharge the value, that is "D" has to be what we want Q to be every tick, limiting the usefulness.

- We need a way to enable or disable the update.



Problems: Owner of a lonely (D) Latch

- We could simply add a signal that we usually keep “0” and only allow the latch when that ‘write enable’ (W) signal is “1”



Let's look at some circuits

- Weird behaviour when clock is used in logic!!
  - ***Note***: N-e-v-e-r, *never ever,* connect logic to a clock port.
    - It's hard to predict the behavior of circuits!

- d_latch.circ

- broken_latch.circ

And The Flip-Flop

The New Problem

- Remember propagation delay? Pesky thing, that.
- Clocks don't *always* help.
  - We sometimes need a clock cycle to compute a value
  - …and then another clock cycle to compute the next thing.
  - …but the next thing needs to be computing the CURRENT thing.
  - …but we would overwrite that input… so it would compute something else
  - …before it was done computing the first thing…
- AHHHHH!!!!!

Waiting for Godata

- We want to record an *intent* to store (latch) a value.
  - That is, to delay the latch by around a cycle.

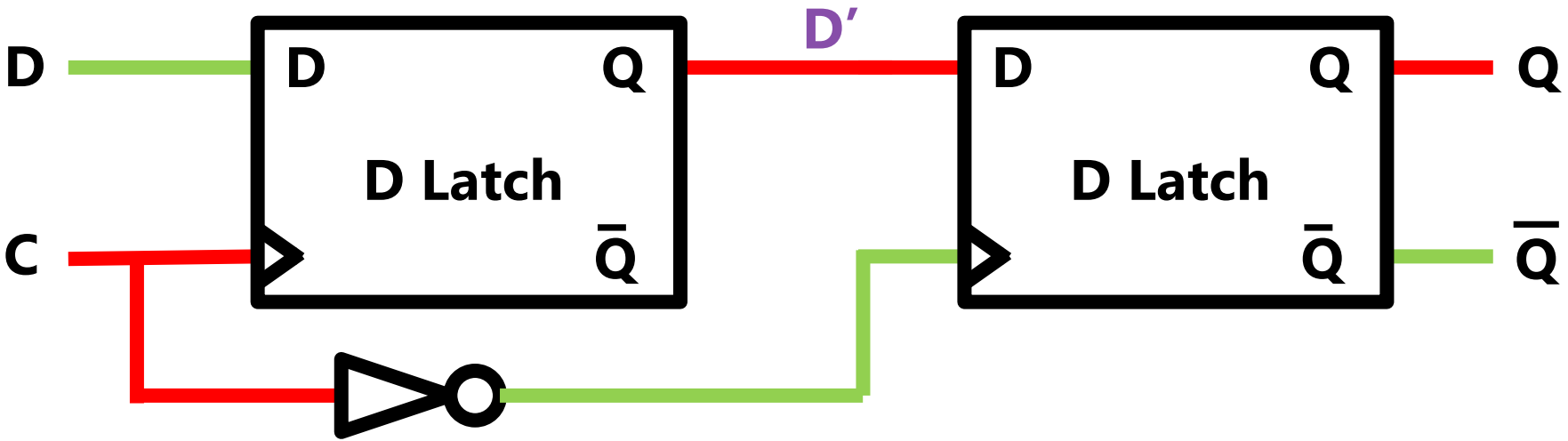
- (But only actually do it at an idle moment)
  - When do we have an idle moment in the latch???
    - When the clock is low!!

- If we *cascade* two D-Latches, and cleverly handle the clock…
  - We can create a register! (Specifically, a D Flip-Flop)
    - Yes, that's actually what it is called. ☺

- We will create a component that latches a value on the clock's *falling* edge.
  - You can also make a rising edge D Flip-Flop by inverting the clock signal.

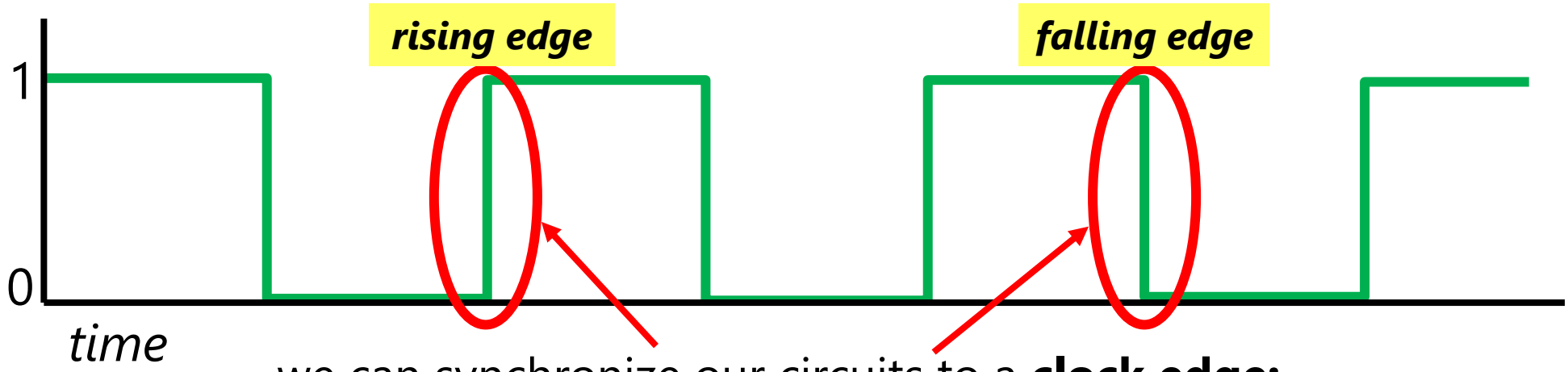
The D Flip-Flop

- While the clock is "1" (high), **D'** can be computed while Q remains unaffected.
  - Q is being used, after all, by whatever component is after the flip-flop
  - While D is not immediately known and is being computed by the component before the flip-flop
- Falling clock edge: value is copied from the first latch to the second.
  - This handles data propagation within a sequential circuit.



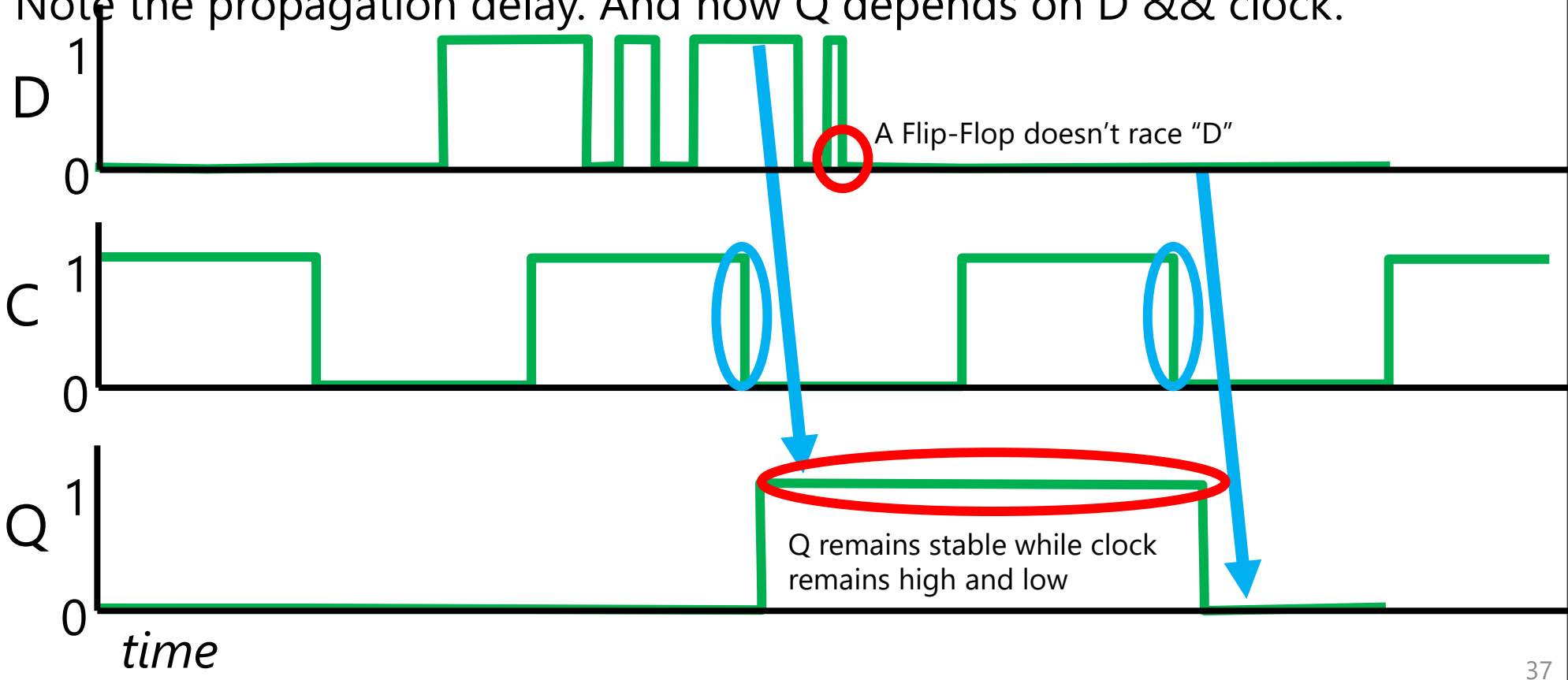
Tick Tock

- In this example, we are using the clock edges
  - The circuit only updates its output in the instant the clock changes!
- During the remaining time, other circuits can compute the values



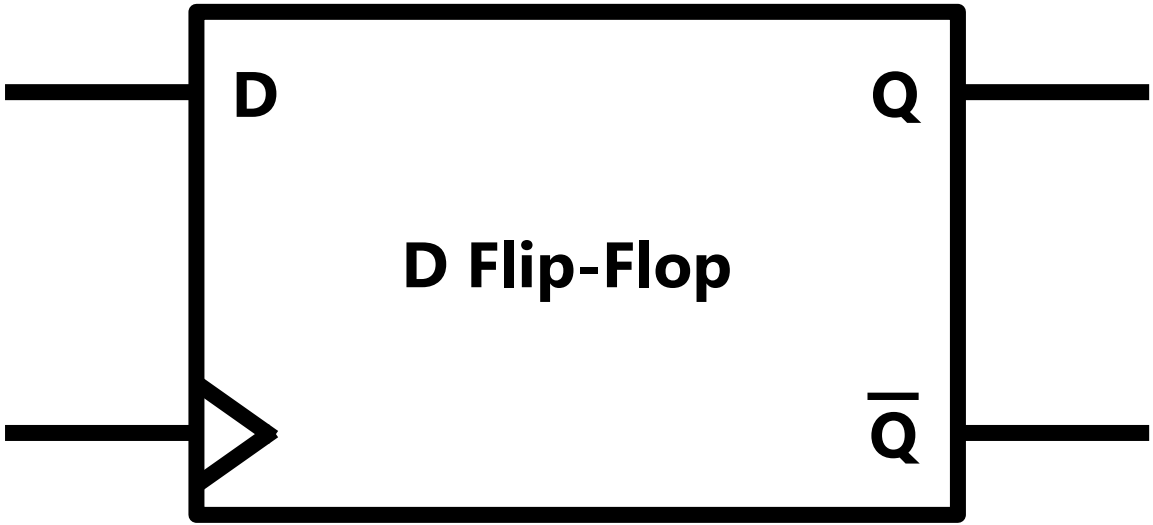
Tick Tock: Falling Edge D Flip-Flop

- This diagram shows the behavior of the system over time.
  - This is "**falling edge**" triggered.
  - Note the propagation delay. And how Q depends on D && clock.



The D Flip-Flop… Abstracted

- We can of course reduce the flip-flop… it looks the same as the D Latch.
  - This is effectively a **1-bit Register**!
  - That is, it is a simple 1-bit volatile memory cell.

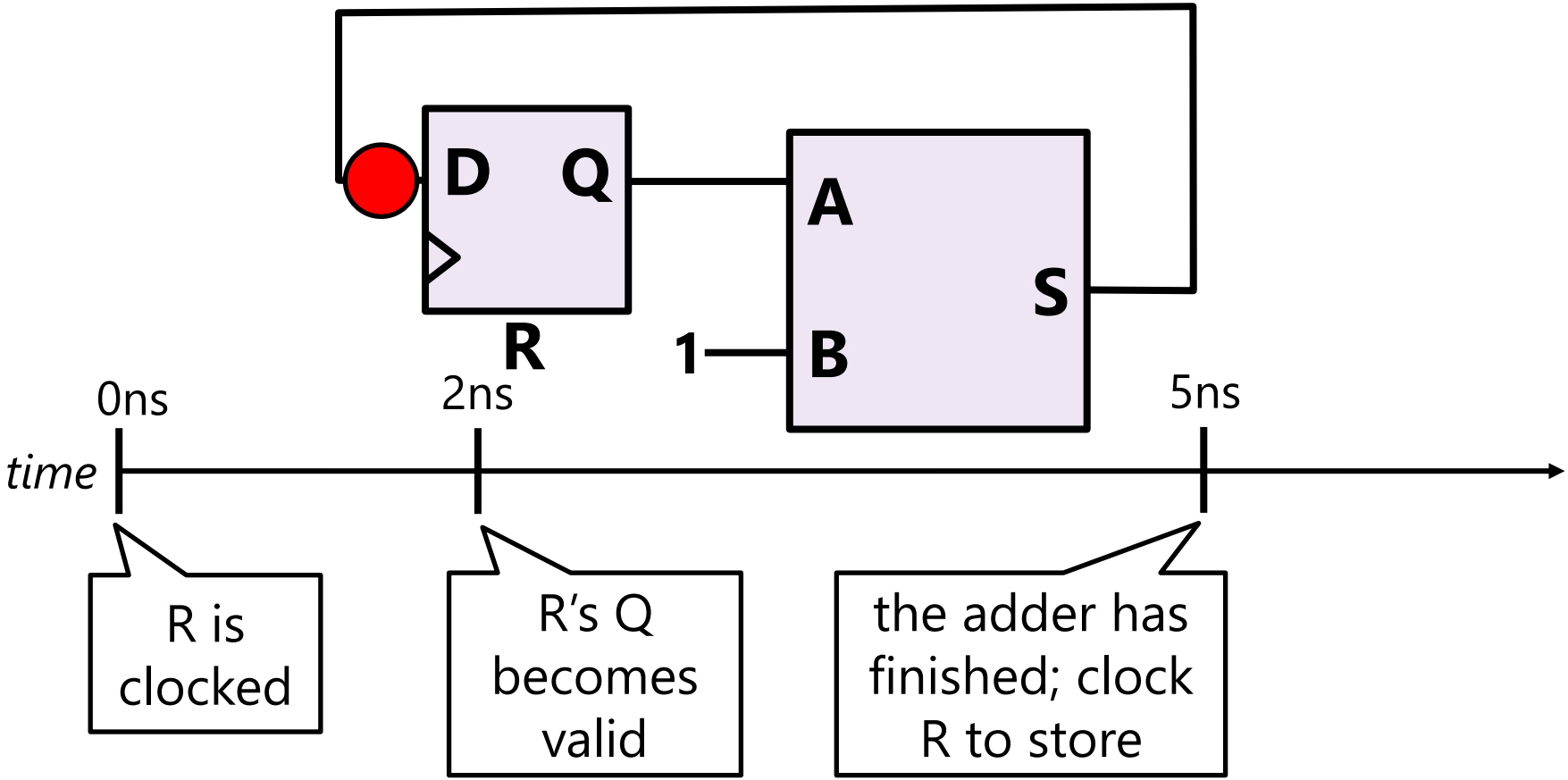


Let's look at some circuits

- D_flip_flop.circ
- broken_latch.circ … again

Real-world clocking issues

Determining clock speed



Determining clock speed

- It takes **5ns** for a signal to propagate through our circuit
- How fast can we clock it?
  - if the time between clocks is **less than 5ns,** we'll clock the register **too early** (while the adder's outputs are **invalid**)
  - if the time between clocks is **more than 5ns,** no big deal

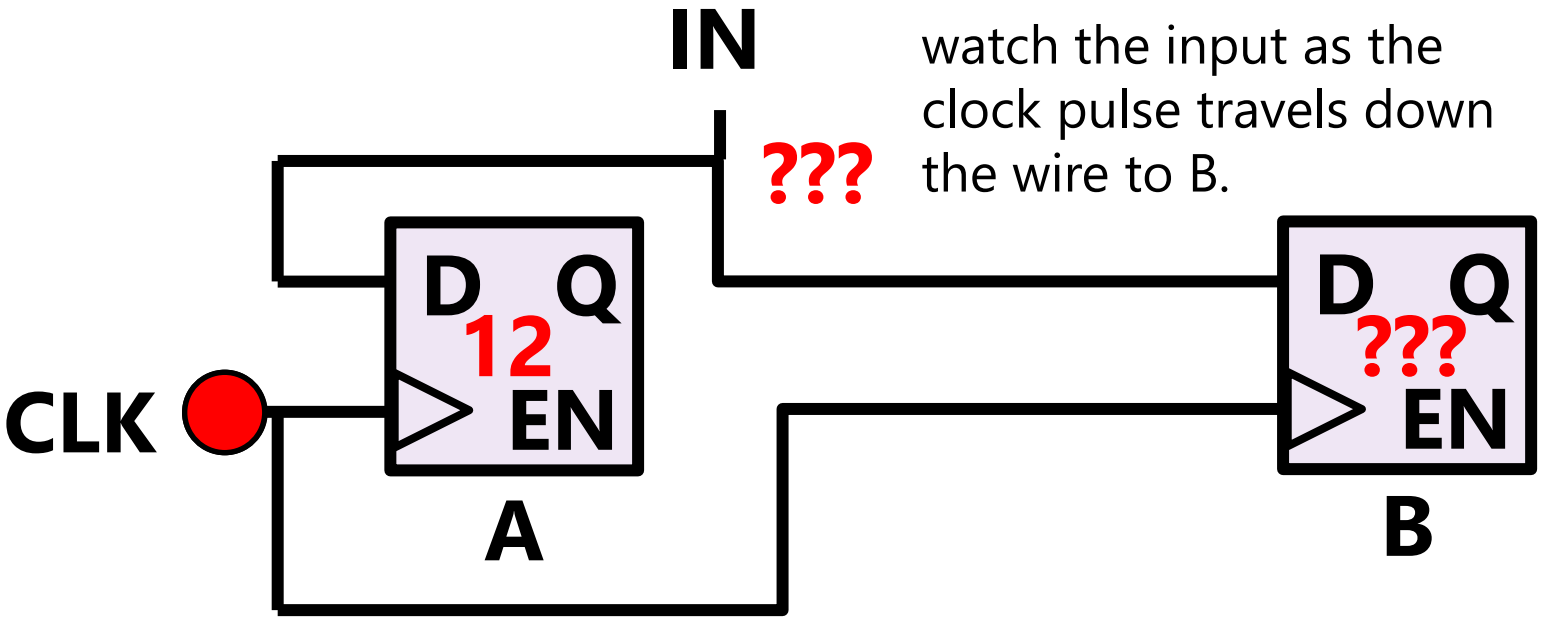
$$\frac{1}{5 \times 10^{-9}\ s} = 0.2 \times 10^{9}\ \text{Hz}$$

$$= 200\text{MHz}$$

- The **fastest we can clock a sequential circuit** is the **reciprocal of the critical path's propagation delay**

Clock Skew

- The *clock signal itself* isn't immune to propagation delay!

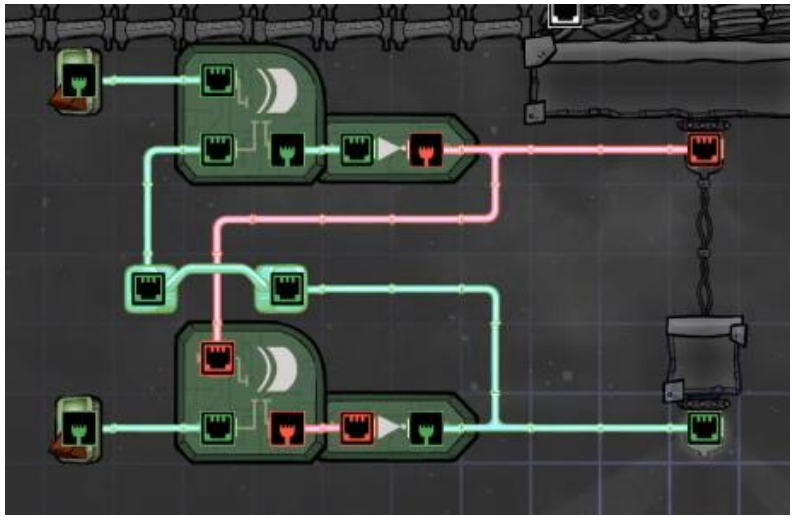


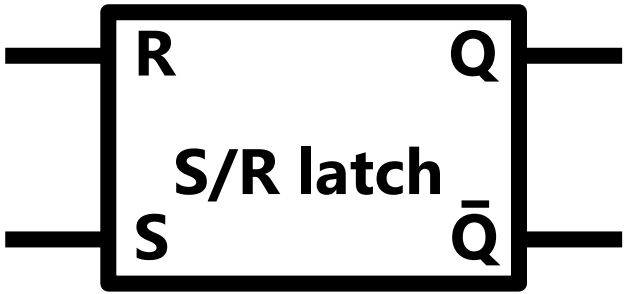
- This is a **race condition:** the data and clock are having a race, and the outcome depends on who wins
  - the winner could change based on temperature, power, etc!

Summary

The S/R Latch … Abstracted

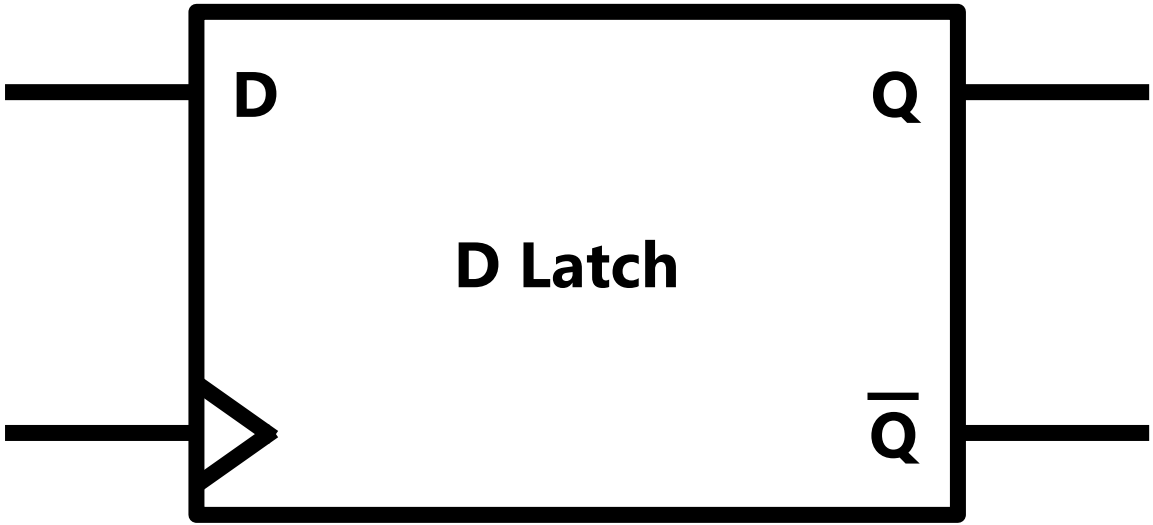
- We now know the S/R Latch!
  - Allows you to store a value (1/0) but with a pit fall!
  - It could turn into an **oscillator** if input an invalid combination
    - **R=S=1 ➔ R=S=0**





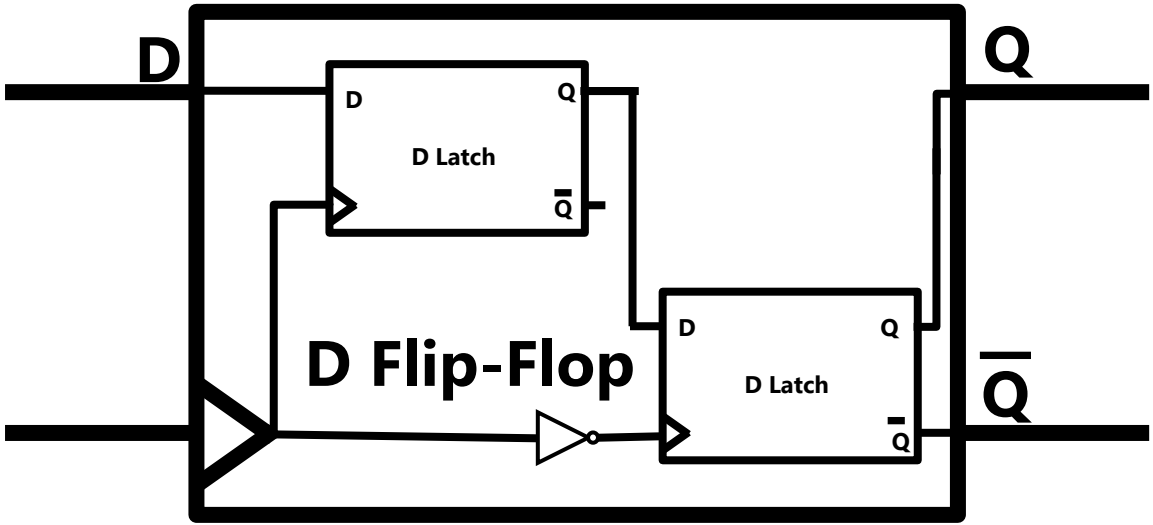
The D Latch … Abstracted

- We improved it by adding some input logic
  - Creating the D Latch
  - But it was transparent (**active**) during the **high** clock state



The D Flip-Flop… Abstracted

- Lastly, we saw the heroic D Flip-Flop!
  - This is effectively a **1-bit Register**!
  - That is, it is a simple 1-bit volatile memory cell.



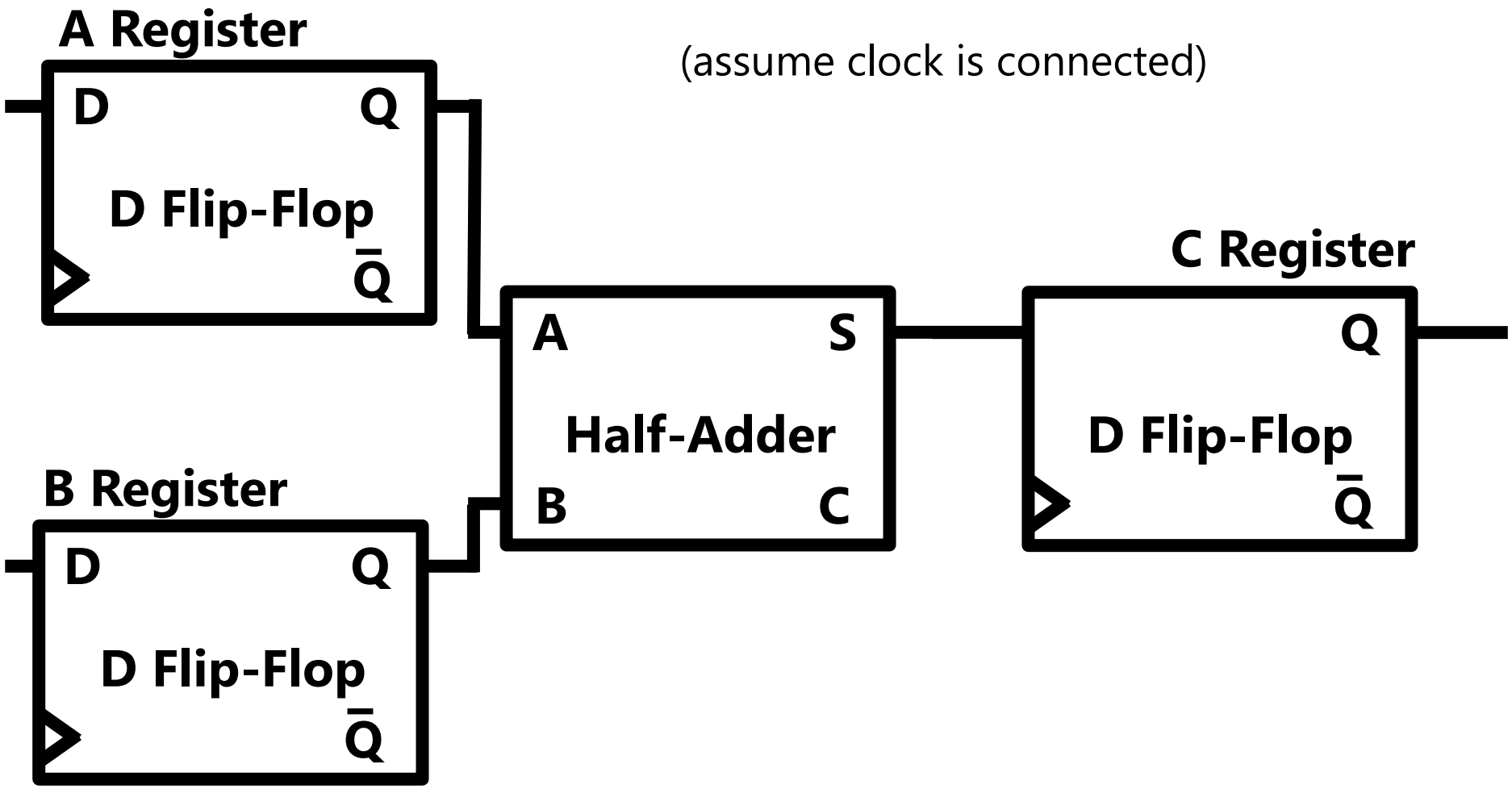
Circuits using Flip-Flops

Creating an Adder Circuit

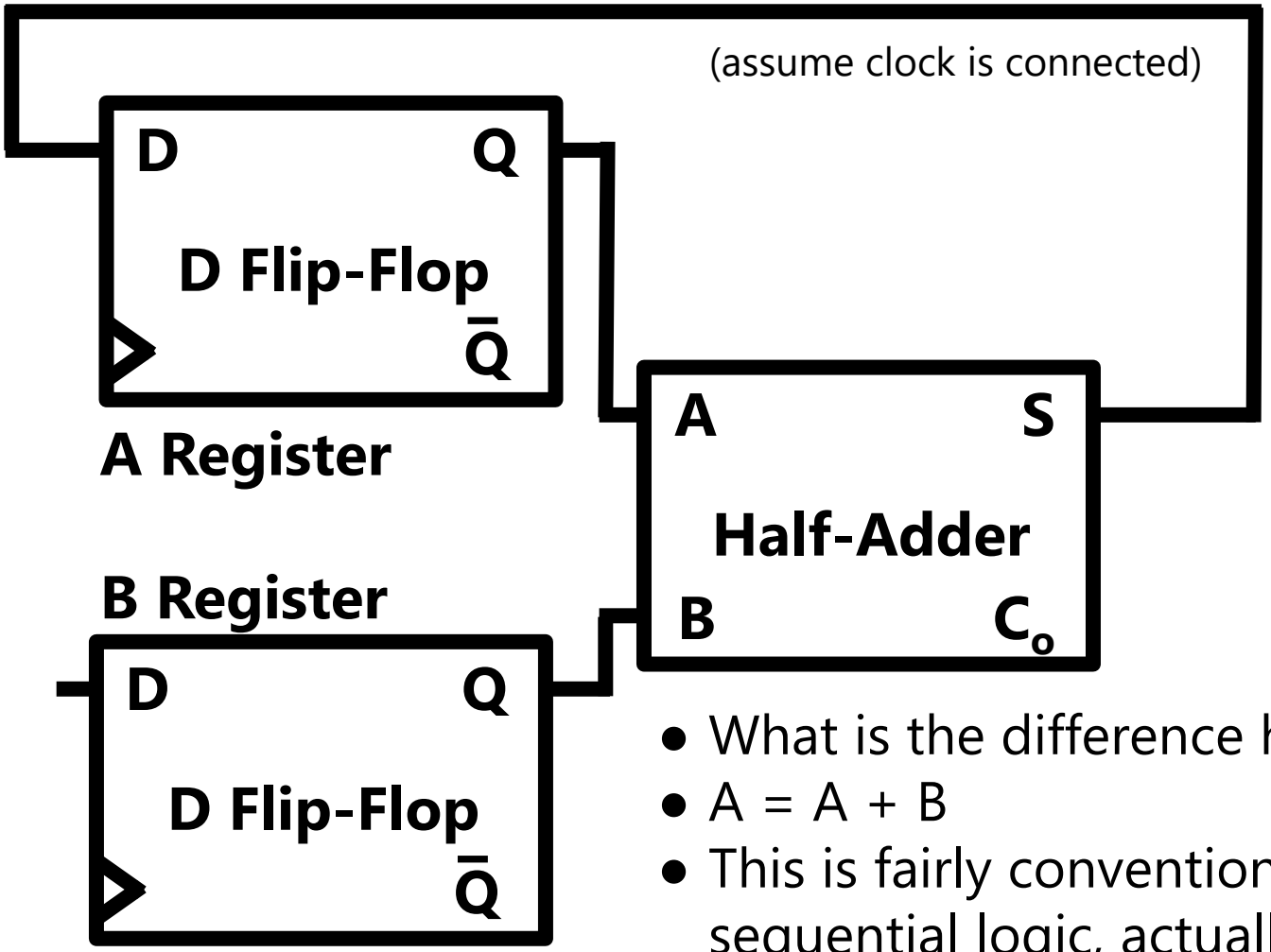
- Suppose we want to (for 1 bit):
  - Have three 1-bit registers: A, B, C
  - Compute: C = A + B

- We would need:
  - Three D Flip-flops (for A, B, and C)
  - A 1-bit adder
  - What is the circuit?

Register-backed 1-bit Adder



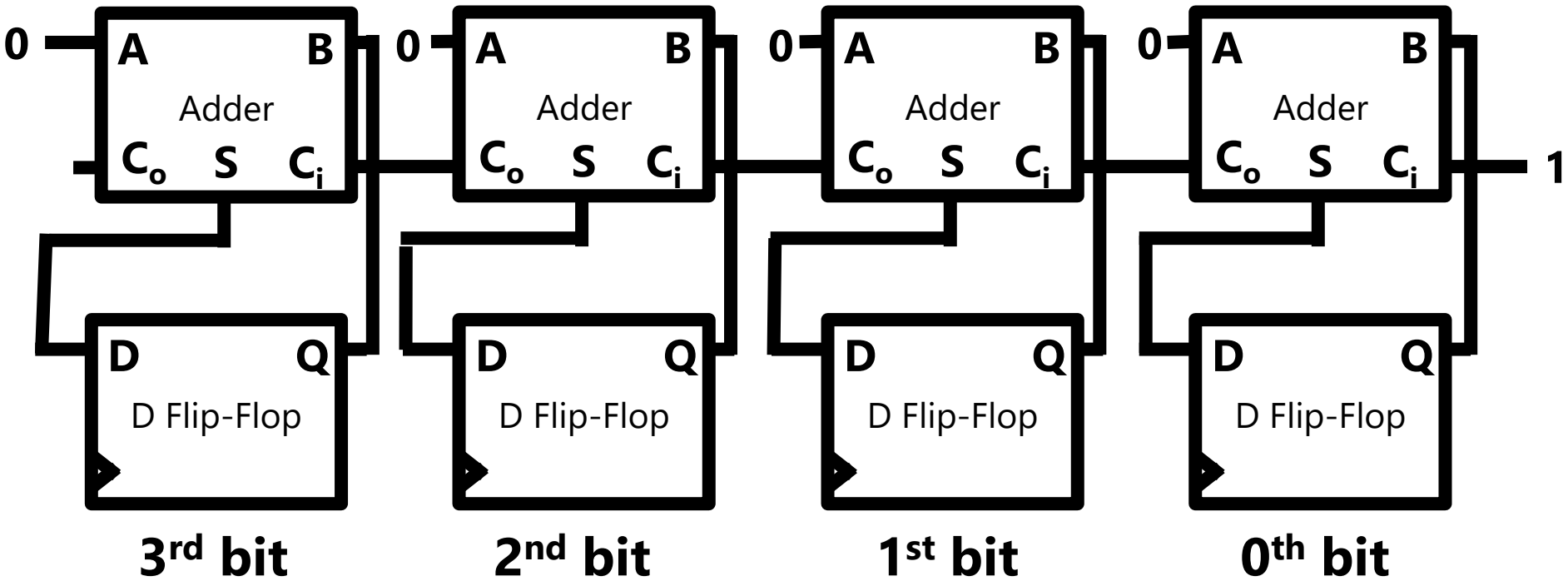
Register-backed 1-bit Adder



- What is the difference here?
- A = A + B
- This is fairly conventional sequential logic, actually.

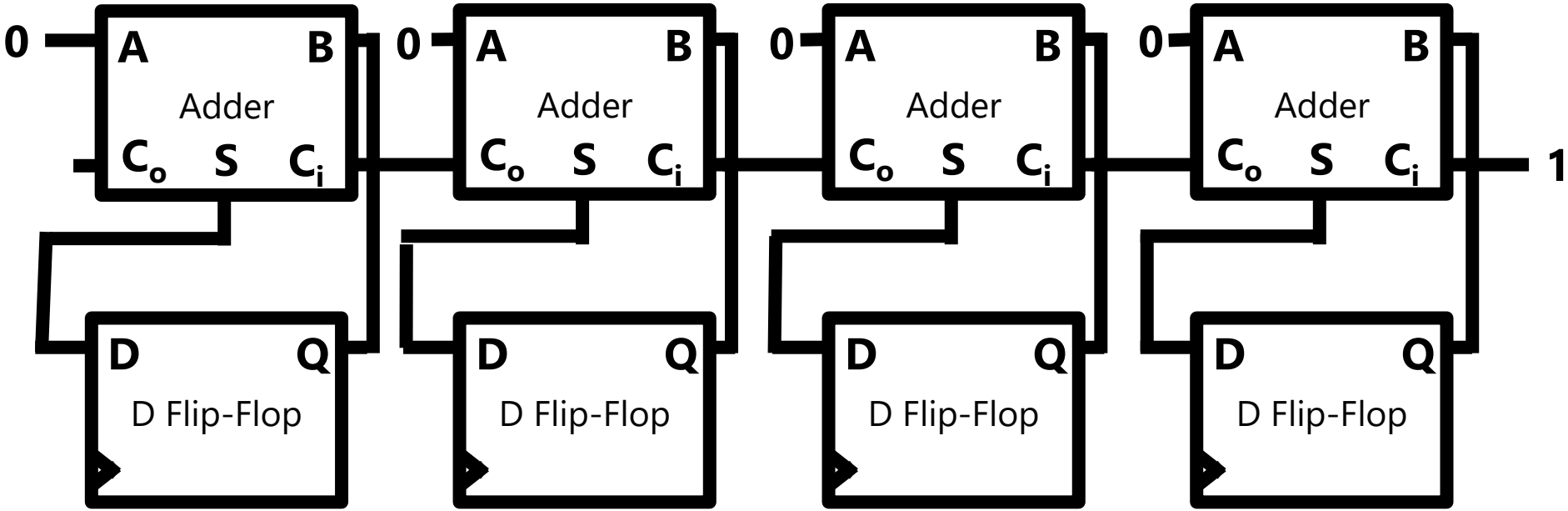
4-bit Counter

- Another simple component we can now build is a counter.
- This is a register that increments every clock tick.
  - On the falling-edge, in this case. (assume clock is connected)

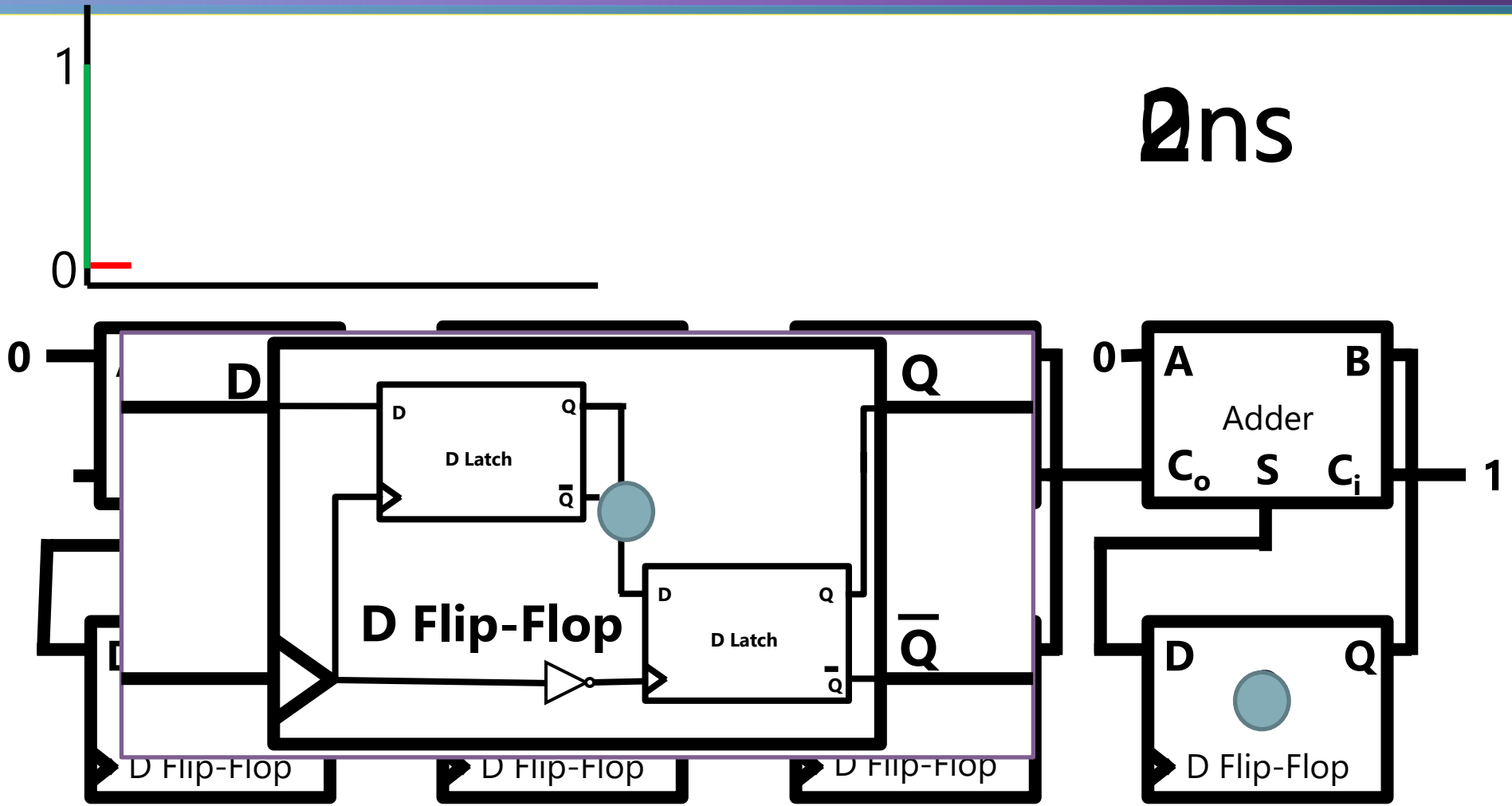


4-bit Counter: Thoughts

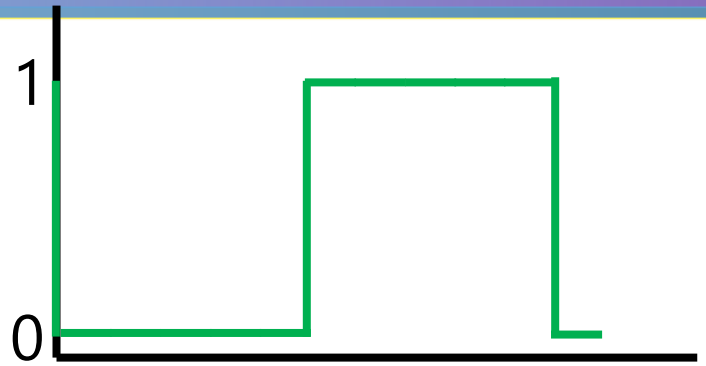
- Hmm, adding takes some time because it *ripples*
- Hmm, how long does our clock cycle have to be?
  - No shorter than the propagation delay.
  - If you assume latches take 2ns and adders take 4ns…



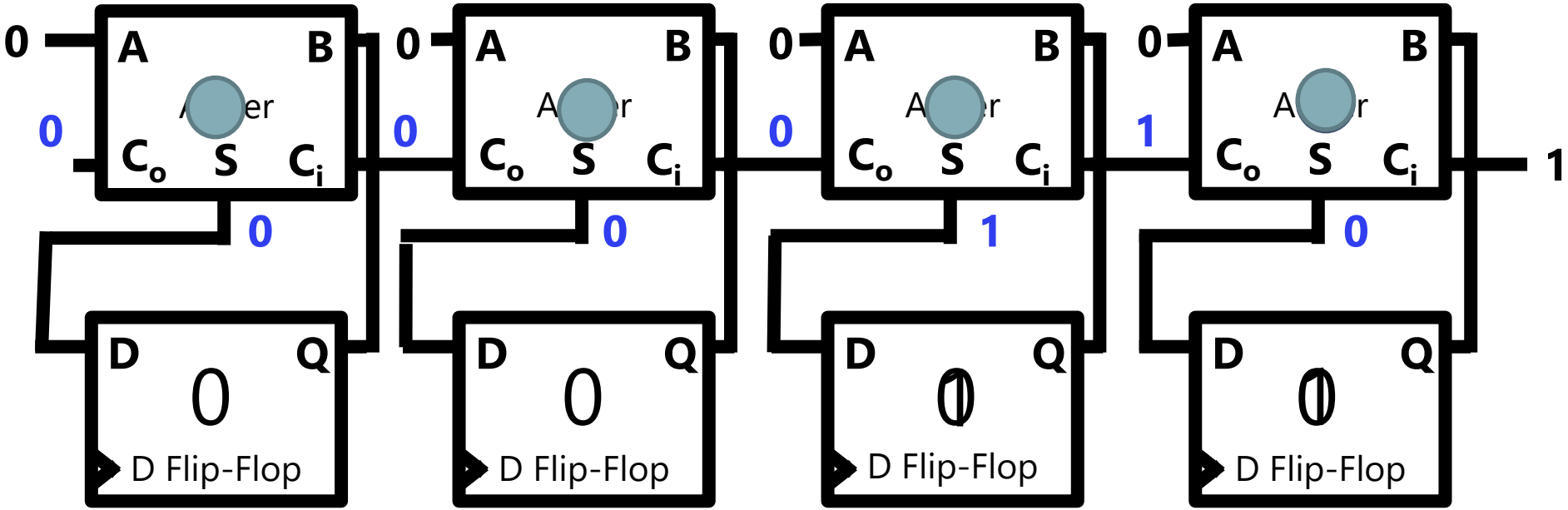
4-bit Counter: Clocking



4-bit Counter: Clocking

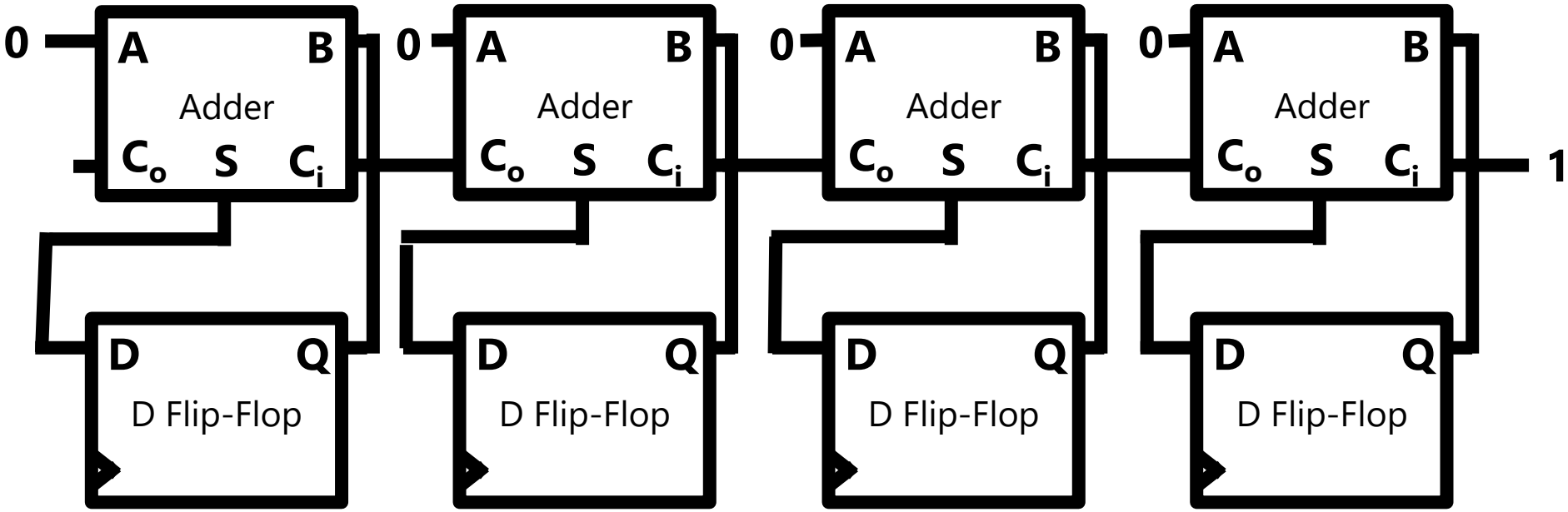


20ns



4-bit Counter: Circuit Delay

Serialized 4ns + 4ns + 4ns + 4ns



Parallelized 2ns + 2ns

20ns

4-bit Counter: Delay Explanation

- Values of output bits ***must all be stable***.
  - That is, can't pulse clock until all 4 bits are computed

- However, the adder is a ripple-carry: Each bit waits for previous.
  - 4ns per adder
  - 4-bit adder
  - Thus: 4 * 4ns = 16ns for the 4-bit adder

- Flip-Flops
  - Must wait for 1st latch to stabilize Q (2ns in parallel)
  - Must wait for 2nd latch to stabilize Q (2ns also in parallel)
  - Thus: 2ns + 2ns = 4ns

- Overall delay: 16ns + 4ns = 20ns. Clock pulse is 20ns.

Determine Maximum Clock Speed

- If this operation takes 20ns, this bounds our clock speed.
- Our clock period must be 20ns.
  - Which means our frequency is once every 20ns: 1s ÷ 20ns
  - What is that frequency in Hz? (Hertz is “cycles per second”)

  - This would be the maximum clock speed for our circuit to work.

$$\frac{1s}{20ns} = 50{,}000{,}000\ Hz = 50\ MHz$$